2 Sa3yda w 1 Menofy

Competitive Programming Reference

2025

Contents

1 Strategy & Notes

1.1 Notes for Hard questions

- Try to brute force values
- get the naive solution → so you can optimize it
- Stress Test
- Test El PC in Practice
- Think in reverse of the problem
- solve the complement and total - cnt is your answer
- if you need to count the ways and the mean is can be computed easier
- ways = expected * total
- Solve range? try to solve(r) - solve(l - 1)

1.2 Checks Before Submission

- Validate (you ask your teammate)
- Time & Memory
- Corner Cases
- Bounding Cases
- Read Clarifications
- check for mod

2 Algebra Formulas

If a and r are real numbers and $r \neq 0$, then

$$\sum_{j=0}^{n} ar^j = \begin{cases} \frac{ar^{n+1}-a}{r-1} & \text{if } r \neq 1 \\ (n+1)a & \text{if } r = 1. \end{cases}$$

$\sum_{k=1}^{n} k$	$\frac{n(n+1)}{2}$
$\sum_{k=1}^{n} k^2$	$\frac{n(n+1)(2n+1)}{6}$
$\sum_{k=1}^{n} k^3$	$\frac{n^2(n+1)^2}{4}$
$\sum_{k=0}^{\infty} x^k, \|x\| < 1$	$\frac{1}{1-x}$
$\sum_{k=1}^{\infty} kx^{k-1}, \|x\| < 1$	$\frac{1}{(1-x)^2}$

- **LCM * GCD** = the multiple of the 2 numbers
- **LCM** = multiply the numbers which has the greatest power from the 2 numbers even if they're not common
- **GCD** = multiply the numbers which has the lowest power from the 2 numbers even if they're not common
- **Arithmetic sequence :** $\frac{n}{2}(a+b)$

- **Geometric sequence :** $\frac{L*R-a}{r-1} = a * \frac{r^n-1}{r-1}$
- **Sum of odd numbers :** $\left(\frac{n+n\%2}{2}\right)^2$
- **Sum of even numbers:** $\frac{\left(\frac{n}{2}*(2+n-n\%2)\right)}{2}$
- **Quadratic Formula :** $\frac{-b\pm\sqrt{b^2-4ac}}{2a}$

3 Main Template

```cpp
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

#define fast ios_base::sync_with_stdio(0);                                    \
    cout.tie(0);                                                              \
    cin.tie(0)

using namespace std;
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag, tree_order_statistics_node_update>;
// order_of_key (k) : Number of items strictly smaller than k
// find_by_order(k) : K-th element in a set (counting from zero)
// const int INF = 1e18;

// Coding is so easy if you simulate on paper first
using lll = __int128_t;


struct chash
{
    static uint64_t splitmix64(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const
    {
        static const uint64_t R = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + R);
    }

    size_t operator()(const pair<uint64_t, uint64_t> &p) const
    {
        return operator()(p.first) ^ (operator()(p.second) << 1);
    }
};

template <typename K>
using hash_set = gp_hash_table<K, null_type, chash>;
```

4 FFT – Templates

4.1 FFT (Complex Double)

```cpp
using cd = complex<double>;
const double pi = acos(-1);

void fft(vector<cd> &a, bool invert)
{
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++)
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1)
    {
        double ang = 2 * pi / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len)
        {
            cd w(1);
            for (int j = 0; j < len / 2; j++)
            {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert)
    {
        for (cd &x : a)
            x /= n;
    }
}

vector<ll> multiply(vector<int> const &a, vector<int> const &b)
{
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + b.size())
        n <<= 1;
```

```
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<ll> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}
```

4.2 Mod FFT

```
typedef complex<double> C;
typedef vector<double> vd;

void fft(vector<C> &a) {
    int n = (int)a.size();
    int L = 31 - __builtin_clz(n);

    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);

    for (static int k = 2; k < n; k *= 2) {
        R.resize(n);
        rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);

        for (int i = k; i < 2 * k; i++)
            rt[i] = R[i] = (i & 1) ? R[i / 2] * x : R[i / 2];
    }

    vector<int> rev(n);

    for (int i = 0; i < n; i++)
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;

    for (int i = 0; i < n; i++)
        if (i < rev[i])
            swap(a[i], a[rev[i]]);

    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                auto x = (double *)&rt[j + k];
                auto y = (double *)&a[i + j + k];
                C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] * y[0]);
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
}

template<int M>
vector<int> convMod(const vector<int> &a, const vector<int> &b) {
    if (a.empty() || b.empty()) return {};

    vector<int> res((int)a.size() + (int)b.size() - 1);

    int B = 32 - __builtin_clz((int)res.size());
    int n = 1 << B;
    int cut = (int)sqrt(M);

    vector<C> L(n), R(n), outs(n), outl(n);

    for (int i = 0; i < (int)a.size(); i++)
        L[i] = C(a[i] / cut, a[i] % cut);

    for (int i = 0; i < (int)b.size(); i++)
        R[i] = C(b[i] / cut, b[i] % cut);

    fft(L);
    fft(R);

    for (int i = 0; i < n; i++) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }

    fft(outl);
    fft(outs);

    for (int i = 0; i < (int)res.size(); i++) {
        long long av = (long long)(real(outl[i]) + .5);
        long long cv = (long long)(imag(outs[i]) + .5);
        long long bv = (long long)(imag(outl[i]) + .5) + (long long)(real(outs[i]) + .5);

        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }

    return res;
}
```

4.3 NTT (mod = 998244353)

```
const int mod = (119 << 23) + 1, root = 3; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.

int fp(ll b, int e)
{
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1)
            ans = ans * b % mod;
    return ans;
}

// Primitive Root of the mod of form 2^a * b + 1
int generator()
{
    vector<int> fact;
    int phi = mod - 1, n = phi;
    for (int i = 2; i * i <= n; ++i)
```

```
        if (n % i == 0)
        {
            fact.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back(n);

    for (int res = 2; res <= mod; ++res)
    {
        bool ok = true;
        for (size_t i = 0; i < fact.size() && ok; ++i)
            ok &= fp(res, phi / fact[i]) != 1;
        if (ok)
            return res;
    }
    return -1;
}

void ntt(vector<int> &a)
{
    int n = (int)a.size(), L = 31 - __builtin_clz(n);
    vector<int> rt(2, 1); // erase the static if you want to use two moduli;
    for (int k = 2, s = 2; k < n; k *= 2, s++)
    { // erase the static if you want to use two moduli;
        rt.resize(n);
        int z[] = {1, fp(root, mod >> s)};
        for (int i = k; i < 2 * k; ++i)
            rt[i] = (ll)rt[i / 2] * z[i & 1] % mod;
    }
    vector<int> rev(n);
    for (int i = 0; i < n; ++i)
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; ++i)
        if (i < rev[i])
            swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
    {
        for (int i = 0; i < n; i += 2 * k)
        {
            for (int j = 0; j < k; ++j)
            {
                int z = (ll)rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
                a[i + j + k] = ai - z + (z > ai ? mod : 0);
                ai += (ai + z >= mod ? z - mod : z);
            }
        }
    }
}
vector<int> conv(const vector<int> &a, const vector<int> &b)
{
    if (a.empty() || b.empty())
        return {};
    int s = (int)a.size() + (int)b.size() - 1, B = 32 - __builtin_clz(s), n = 1 << B;
    int inv = fp(n, mod - 2);
    vector<int> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    for (int i = 0; i < n; ++i)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}
```

4.4 CRT (3-mod combination)

```
int mod, root, desired_mod = 1000000007;
const int mod1 = 167772161;
const int mod2 = 469762049;
const int mod3 = 754974721;
const int root1 = 3;
const int root2 = 3;
const int root3 = 11;

int CRT(int a, int b, int c, int m1, int m2, int m3) {
    __int128 M = (__int128)m1*m2*m3;
    ll M1 = (ll)m2*m3;
    ll M2 = (ll)m1*m3;
    ll M3 = (ll)m2*m1;

    int M_1 = modpow(M1%m1, m1 - 2, m1);
    int M_2 = modpow(M2%m2, m2 - 2, m2);
    int M_3 = modpow(M3%m3, m3 - 2, m3);

    __int128 ans = (__int128)a*M1*M_1;
    ans += (__int128)b*M2*M_2;
    ans += (__int128)c*M3*M_3;

    return (ans % M) % desired_mod;
}
```

5 FFT – Problems

5.1 A+B via FFT

```
void solve()
{
    int n;
    cin >> n;
    vector<int> a(n);
    int mn = 0;
    for (int i = 0; i < n; i++)
        cin >> a[i], mn = min(a[i], mn);
    const int shift = 5e4;
    vector<int> p1(4 *shift + 1);
    int zeros = 0;
    for (int i = 0; i < n; i++)
        p1[a[i] + shift]++, zeros += (a[i] == 0);
    auto pw = multiply(p1, p1);
    for (int i = 0; i < n; i++)
            pw[2 * a[i] + 2 * shift]--;
    int ans = 0;
    for (int i = 0; i < n; i++)
    {

```

```
        ans += pw[a[i] + 2 * shift] - 2 * (zeros - (a[i] == 0));
    }
    cout << ans << endl;
}
```

5.2 A-B Convolution

```
vector<int> A_Minus_B(vector<int> a, vector<int> b)
{
    int shift = b.size() - 1;
    reverse(all(b));
    // or
    // for (int i = 0; i < b.size() / 2; i++)
    //     swap(b[i], b[shift - i]);
    vector<int> ans = conv(a, b);
    return {ans.begin() + shift, ans.end()};
}
```

5.3 Poly Shift

```
vector<int> poly_shift(vector<int> &p, int k)
{
    int deg = p.size() - 1;
    vector<int> p1(deg + 1), p2(deg + 1);
    for (int i = 0; i <= deg; i++)
    {
        p1[i] = mul(p[i], fact[i]);
        p2[deg - i] = mul(fp(k, i), ifact[i]);
    }
    auto res = conv(p1, p2);
    vector<int> ans(deg + 1);
    for(int i= 0;i <= deg; i++)
        ans[i] = mul(res[i + deg], ifact[i]);
    return ans;
}
```

5.4 Multi Polynomial Multiply

```
void solve()
{
    int m;
    cin >> m;
    vector<vector<ll>> polys(m + 1);
    priority_queue<pii,vector<pii>, greater<>> pq; // (size, idx)
    polys[0] = {1};
    pq.push({1, 0});
    for(int i = 1; i <= m; i++)
    {
        int n;
        cin >> n;
        polys[i].resize(n + 1);
        for(auto &x : polys[i])
            cin >> x;
        pq.push({n + 1, i});
    }
    while(pq.size() > 1)
    {
        int nIdx = pq.top().second;
        auto &p1 = polys[pq.top().second];
        pq.pop();
        auto &p2 = polys[pq.top().second];
        pq.pop();
        int nSz = p1.size() + p2.size() - 1;
        p1 = conv(p1, p2);
        pq.push({nSz, nIdx});
    }
    auto &ans = polys[pq.top().second];
    for(auto & x :ans)
        cout << x << ' ';
}
```

5.5 Counting Distinct Array

```
// You have some elements the max ouccurace of each element is m and you want to generate
    array of size of k of this elements
// the answer is fact[k] * sum(ifact of all possible frequencies of each element using ntt or
    fft with mod)

void solve()
{
    int n, m;
    cin >> n >> m;
    vector<int> p(m + 1);
    p[m] = fp(fact[m], mod - 2);
    for (int i = m - 1; i >= 0; i--)
        p[i] = 1LL * p[i + 1] * (i + 1) % mod;
    p = poly_pw(p, n);
    int q;
    cin >> q;
    while (q--)
    {
        int k;
        cin >> k;
        cout << (1LL * fact[k] * p[k] % mod)<<' ';
    }
}
```

5.6 Product Modulo

```
// sum (a[i] * b[i] % mod) but sum is not under mod

const int P = 200003;
int to[P];
void solve()
{
    ll cur = 1;
    for (int i = 0; i < P; i++, (cur *= 2) %= P)
        to[cur] = i;
    vector<int> p(P);
    ll sum = 0;
    int n;
    cin >> n;
    for (int i = 0, x; i < n; i++)
        cin >> x, p[to[x]] += (x > 0), sum -= 1ll * x * x % P;
    auto ans = multiply(p, p);
    for (int i = 0; i < ans.size(); i++)
        if (ans[i])
```

```
            sum += 1ll * fp(2, i, P) * ans[i];
    cout << sum / 2 << endl;
}
```

5.7 String Matching

```
string chars = "abcdefghijklmnopqrstuvxyz";

int match_all_chars(const string &T, const string &P)
{
    int n = T.size(), m = P.size();
    if (m > n)
        return {};

    int required_matches = 0;
    for (char c : P)
    {
        if (c != '?')
            required_matches++;
    }

    vector<int> total_matches(n + m, 0);

    // Loop over all possible alphabet characters
    for (char c = 'a'; c <= 'z'; c++)
    {
        // Skip character if not present in P to save FFT operations
        bool in_p = false;
        for (char pc : P)
        {
            if (pc == c)
            {
                in_p = true;
                break;
            }
        }
        if (!in_p)
            continue;

        vector<int> A(n, 0), B(m, 0);
        for (int i = 0; i < n; i++)
            if (T[i] == c)
                A[i] = 1;
        for (int i = 0; i < m; i++)
            if (P[m - 1 - i] == c)
                B[i] = 1;

        vector<ll> C = multiply(A, B);
        for (int i = 0; i < n + m; i++)
        {
            total_matches[i] += C[i];
        }
    }

    int cnt = 0;
    for (int i = 0; i <= n - m; i++)
        cnt += (total_matches[i + m - 1] == required_matches);

    return cnt;
}
```

5.8 String Matching WildCard

```
void solve(int tc) {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<cd> poly1(n), poly2(m);

    for (int i = 0; i < n; ++i) {
        double angle = 2*PI*(s[i]-'a')/26;
        poly1[i] = cd(cos(angle), sin(angle));
    }
    for (int i = 0; i < m; ++i) {
        if(patt[m-i-1] == '*') poly2[i] = cd(0,0); // Wild Card
        else {
            double angle = 2*PI*(patt[m-i-1]-'a')/26;
            poly2[i] = cd(cos(angle), -sin(angle));
        }
    }

    vector<cd> ans = multiply(poly1, poly2);
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');

    int tot = 0;
    vector<int> pos;
    for (int i = 0; i < n; ++i) {
        if(fabs(ans[m-1+i].real() - (m - wild_cnt)) < eps && fabs(ans[m-1+i].imag()) < eps) {
            ++tot;
            pos.push_back(i);
        }
    }

    cout << tot << "\n";
    for(auto & p : pos) cout << p << " ";
    cout << "\n";

}
```

5.9 String Min Mismatch

```
const char chrs[] = {'A', 'C', 'T', 'G'};
int stringMinMismatch(string str, string pattern)
{
    int n = str.size();
    int m = pattern.size();
    int shift = m - 1;
    vector<int> ans(n + m + 5);
    for (auto &ch : chrs)
    {
        vector<int> p1(n);
        vector<int> p2(m);
        for (int i = 0; i < n; i++)
            p1[i] = str[i] == ch;
        for (int i = 0; i < m; i++)
            p2[shift - i] = pattern[i] == ch;
        auto mult = multiply(p1, p2);
```

```
        for (int i = 0; i < n; i++)
            ans[i + shift] += mult[i + shift];
    }
    int mn = m;
    for (int i = 0; i <= n - m; i++)
        mn = min(mn, m - ans[i + shift]);
    return mn;
}
void solve()
{
    string s, p;
    cin >> s >> p;
    cout << stringMinMismatch(s, p) << endl;
}
```

5.10 Fuzzy Search

```
const string chars = "ATGC";

void solve()
{
    int n, m, k;
    cin >> n >> m >> k;
    string s, t;
    cin >> s >> t;
    int shift = m - 1;
    vector<int> ans(n);
    for (auto &ch : chars)
    {
        vector<int> p1(n + 1), p2(shift + 1);
        int cnt = 0;

        for (int i = 0; i < n; i++)
            if (s[i] == ch)
                p1[max(0, i - k)]++, p1[min(n, i + k + 1)]--, cnt++;
        for (int i = 0; i < n; i++)
            p1[i + 1] += p1[i];
        p1.pop_back();
        for (int i = 0; i < n; i++)
            p1[i] = bool(p1[i]);
        for (int i = 0; i < m; i++)
            p2[shift - i] = t[i] == ch;
        auto mult = multiply(p1, p2);
        for (int i = 0; i < n; i++)
            ans[i] += mult[i + shift];
    }
    cout << count(all(ans), m) << endl;
}
```

5.11 Max Self Matching

```
const char chrs[] = {'a', 'b', 'c'};
vector<int> stringMaxSelfMatching(string s)
{
    string p = s;
    int n = s.size();
    s = s + string(n, '#');
    int shift = n - 1;
    vector<int> ans(2 * n + 5);
    for(auto &ch : chrs)
    {
        vector<int> p1(2 * n);
        vector<int> p2(n);
        for(int i = 0; i < 2 * n; i++)
            p1[i] = s[i] == ch;
        for(int i = 0; i < n; i++)
            p2[shift - i] = s[i] == ch;
        auto mult = multiply(p1, p2);
        for(int i = 1;i < n; i++)
            ans[i + shift] += mult[i + shift];
    }
    return vector<int>(ans.begin() + shift, ans.begin() + shift + n);
}
void solve()
{
    string s;
    cin >> s;
    auto ans = stringMaxSelfMatching(s);
    int mx = *max_element(all(ans));
    cout << mx << endl;
    for(int i = 1; i < s.size(); i++)
        if(ans[i] == mx)
            cout << i << endl;
}
```

6 Data Structures

6.1 BIT (Fenwick Tree)

```
struct BIT {
    int n;
    vector<int> tree;

    BIT(int n) : n(n) {
        tree.resize(n + 1);
    }

    void update(int idx, int delta) {
        for (; idx <= n; idx += (idx & -idx))
            tree[idx] += delta;
    }

    int pref(int idx) {
        int ret = 0;
        for (; idx > 0; idx -= (idx & -idx))
            ret += tree[idx];
        return ret;
    }

    int get(int l, int r) {
        return pref(r) - pref(l - 1);
    }
};
```

6.2 Template BIT

```cpp
template <typename T>
class FenwickTree
{
public:
   vector<T> tree;
   int n;
   void init(int n)
   {
      tree.assign(n + 2, 0);
      this->n = n;
   }
   T merge(T &x, T &y) { return x + y; }
   void update(int x, T val)
   {
      for (; x <= n; x += x & -x)
      {
         tree[x] = merge(tree[x], val);
      }
   }
   T getPrefix(int x)
   {
      if (x <= 0)
         return 0;
      T ret = 0;
      for (; x; x -= x & -x)
      {

         ret = merge(ret, tree[x]);
      }
      return ret;
   }
   T getRange(int l, int r)
   {
      return getPrefix(r) - getPrefix(l - 1);
   }
   int lowerBound(ll x)
   {
      int pos = 0;
      for (int sz = (1 << __lg(n)); sz > 0 && x; sz >>= 1)
      {
         if (pos + sz <= n && tree[pos + sz] < x)
         {
            x -= tree[pos + sz];
            pos += sz;
         }
      }
      return pos + 1;
   }
};
```

6.3 2D BIT

```cpp
template <typename T>
class FenwickTree2D
{
public:
    vector<vector<T>> tree;
    int n, m;
    void init(int n, int m)
    {
        tree.assign(n + 2, vector<T>(m + 2, 0));
        this->n = n;
        this->m = m;
    }
    T merge(T &x, T &y) { return x + y; }
    void update(int x, int y, T val)
    {
        for (; x <= n; x += x & -x)
        {
            for (int z = y; z <= m; z += z & -z)
            {
                tree[x][z] = merge(tree[x][z], val);
            }
        }
    }
    T getPrefix(int x, int y)
    {
        if (x <= 0)
            return 0;
        T ret = 0;
        for (; x; x -= x & -x)
        {
            for (int z = y; z; z -= z & -z)
            {
                ret = merge(ret, tree[x][z]);
            }
        }
        return ret;
    }
    T getSquare(int xl, int yl, int xr, int yr)
    {
        return getPrefix(xr, yr) + getPrefix(xl - 1, yl - 1) -
               getPrefix(xr, yl - 1) - getPrefix(xl - 1, yr);
    }
};
```

6.4 Range BIT

```cpp
struct BIT
{
    vector<int> tree;
    vector<int> tree2;
    int treeSize;
    BIT(int n) : treeSize(n)
    {
        tree.assign(n + 1, 0);
        tree2.assign(n + 1, 0);
    }
    void add(int idx, int delta, vector<int> &tree)
    {
        while (idx <= treeSize)
        {
            tree[idx] += delta;
            idx += (idx & -idx);
        }
    }
    int query(int idx, vector<int> &tree)
    {
```

```
        int ret = 0;
        while (idx > 0)
        {
            ret += tree[idx];
            idx -= (idx & -idx);
        }
        return ret;
    }
    void update(int l, int r, int delta)
    {
        add(l, delta, tree);
        add(r + 1, -delta, tree);
        add(l, delta * (l - 1), tree2);
        add(r + 1, -delta * r, tree2);
    }
    int pref(int idx)
    {
        return query(idx, tree) * idx - query(idx, tree2);
    }
    int get(int l, int r)
    {
        return pref(r) - pref(l - 1);
    }
};
```

6.5 Segment Tree (lazy)

```
struct Node
{
    ll val = 0;
};
struct SegmentTree
{
    Node neutral;
    vector <Node> tree;
    vector <ll> lazy;
    SegmentTree(int n)
    {
        tree.resize(4 * n);
        lazy.resize(4 * n, 1e6);
    }
    Node single(int idx, vector <int> &a)
    {
        return Node({a[idx]});
    }
    Node merge(Node x, Node y)
    {
        return Node({x.val + y.val});
    }
    void build(int node, int s, int e, vector <int> &a)
    {
        if(s == e)
        {
            tree[node] = single(s, a);
            return;
        }

        int m = (s + e) >> 1;

        build(2 * node, s, m, a);
        build(2 * node + 1, m + 1, e, a);
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }

    void propagate(int node, int s, int e)
    {
        if(lazy[node] == (ll)1e6)
            return;
        if(s == e)
        {
            tree[node].val = aa[s + lazy[node]];
        }
        if(s != e)
        {
            lazy[2 * node] = lazy[node];
            lazy[2 * node + 1] = lazy[node];
        }
        lazy[node] = 1e6;
    }
    void update(int node, int s, int e, int l, int r, int val)
    {
        propagate(node, s, e);
        if(s > r || e < l)
            return;
        if(s >= l && e <= r)
        {
            lazy[node] = val;
            propagate(node, s, e);
            return;
        }
        int m = (s + e) >> 1;

        update(2 * node, s, m, l, r, val);
        update(2 * node + 1, m + 1, e, l, r, val);
        tree[node] = merge(tree[2 * node], tree[2 * node + 1]);
    }

    Node query(int node, int s, int e, int l, int r)
    {
        propagate(node, s, e);
        if(s > r || e < l)
            return neutral;
        if(s >= l && e <= r)
            return tree[node];
        int m = (s + e) >> 1;
        return merge(query(2 * node, s, m, l ,r), query(2 * node + 1, m + 1, e, l, r));
    }
};
```

6.6 Iterative Segment Tree

```
template <typename T>
struct SegmentTree
{
    int treeSize;
    vector<T> tree;
    T neutral = T();
    SegmentTree(int n, vector<int> arr = {})
```

```
    {
        treeSize = 1;
        while (treeSize < n)
            treeSize <<= 1;
        tree.assign((treeSize << 1), neutral);
        for (int i = 0; i < arr.size(); i++)
            tree[i + treeSize] = T(arr[i]);
        for (int i = treeSize - 1; i > 0; i--)
            tree[i] = tree[i << 1] + tree[i << 1 | 1];
    }
    inline void update(int idx, int v) // idx : (0-based)
    {
        idx += treeSize;
        tree[idx].change(v);
        for (idx >>= 1; idx > 0; idx >>= 1)
            tree[idx] = tree[idx << 1] + tree[idx << 1 | 1];
    }
    inline int get(int l, int r) // L : (inclusive), R : (exclusive)
    {
        T lf, rf;
        for (l += treeSize, r += treeSize; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
                lf = lf + tree[l++];
            if (r & 1)
                rf = tree[--r] + rf;
        }
        lf = lf + rf;
        return lf;
    }
};
struct Node
{
    int sum;
    Node(int sum = INF) : sum(sum)
    {
    }
    inline void change(int sum)
    {
        this->sum = sum;
    }
    operator int() const
    {
        return this->sum;
    }
    const Node operator+(const Node &other)
    {
        return Node(min(this->sum, other.sum));
    }
};
```

6.7 Max Subarray Sum Seg Tree

```
template <typename T>
struct SegmentTree
{
    int treeSize;
    vector<T> tree;
    T neutral = T();
    SegmentTree(int n, vector<int> arr = {})
    {
        treeSize = 1;
        while (treeSize < n)
            treeSize <<= 1;
        tree.assign((treeSize << 1), neutral);
        for (int i = 0; i < arr.size(); i++)
            tree[i + treeSize] = T(arr[i]);
        for (int i = treeSize - 1; i > 0; i--)
            tree[i] = tree[i << 1] + tree[i << 1 | 1];
    }
    inline void update(int idx, int v) // idx : (0-based)
    {
        idx += treeSize;
        tree[idx].change(v);
        for (idx >>= 1; idx > 0; idx >>= 1)
            tree[idx] = tree[idx << 1] + tree[idx << 1 | 1];
    }
    inline int get(int l, int r) // L : (inclusive), R : (exclusive)
    {
        T lf, rf;
        for (l += treeSize, r += treeSize; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
                lf = lf + tree[l++];
            if (r & 1)
                rf = tree[--r] + rf;
        }
        lf = lf + rf;
        return lf;
    }
};
struct Node
{
    int mx, sum, suff, pref;
    Node(int sum = 0)
    {
        this->sum = sum;
        sum = max(0ll, sum);
        this->mx = this->suff = this->pref = sum;
    }
    inline void change(int sum)
    {
        this->sum = sum;
        sum = max(0ll, sum);
        this->mx = this->suff = this->pref = sum;
    }
    operator int() const
    {
        return this->mx;
    }
    const Node operator+(const Node &other)
    {
        Node ret;
        ret.sum = sum + other.sum;
        ret.pref = max(pref, sum + other.pref);
        ret.suff = max(other.suff, suff + other.sum);
        ret.mx = max({
            mx,
```

```
            other.mx,
            suff + other.pref
        });
        return ret;
    }
};
```

6.8 Persistent Segment Tree

```
struct Node
{
    int sum;
    // int lx, rx; // lx (inclusive), rx(exclusive)
    Node *l, *r;
    Node() : sum(0), l(NULL), r(NULL)
    {
    }
    Node(int sum) : sum(sum), l(NULL), r(NULL)
    {
    }
};
const int R = 1e9 + 1;
int mul(int a, int b)
{
    return (a * b) % mod;
}
void merge(Node *ret)
{
    ret->sum = 1;
    ret->sum = mul(ret->sum, ret->l ? ret->l->sum : 0);
    ret->sum = mul(ret->sum, ret->r ? ret->r->sum : 0);
}
Node *merge(Node *lf, Node *rf)
{
    Node *ret = NULL;
    if (!lf and !rf)
        return NULL;
    ret = new Node();
    ret->sum = 1;
    if (lf)
    {
        ret->sum = mul(ret->sum, lf->sum);
        delete lf;
        lf = NULL;
    }
    if (rf)
    {
        ret->sum = mul(ret->sum, rf->sum);
        delete rf;
        rf = NULL;
    }
    return ret;
}
Node *root;
void upd(int idx, int delta, Node *cur, int lx, int rx)
{

    if (rx - lx == 1)
    {
        cur->sum += delta;
        return;
    }
    int mid = (lx + rx) / 2;
    if (idx < mid)
    {
        if (!cur->l)
            cur->l = new Node();
        upd(idx, delta, cur->l, lx, mid);
    }
    else
    {
        if (!cur->r)
            cur->r = new Node();
        upd(idx, delta, cur->r, mid, rx);
    }
    merge(cur);
}
void upd(int idx, int delta)
{
    upd(idx, delta, root, 0, R);
}
Node *get(int l, int r, Node *cur, int lx, int rx)
{
    if (lx >= r or rx <= l)
        return NULL;
    if (!cur)
        return new Node(0);
    if (l <= lx and rx <= r)
        return new Node(cur->sum);
    int mid = (lx + rx) / 2;
    auto lf = get(l, r, cur->l, lx, mid);
    auto rf = get(l, r, cur->r, mid, rx);
    auto ret = merge(lf, rf);
    return ret;
}
int get(int l, int r)
{
    Node *ret = get(l, r, root, 0, R);
    if (!ret)
        return 0;
    int ret_sum = ret->sum;
    delete ret;
    ret = NULL;
    return ret_sum;
}
void clear(Node *cur)
{
    if (cur->l)
        clear(cur->l);
    if (cur->r)
        clear(cur->r);
    delete cur;
    cur = NULL;
}
```

6.9 Sparse Table

```
ll myFunction(ll a,ll b){
```

```
    return a;
}
struct SparseTable {
    vector<vector<ll>> table;
    vector<int> lgs;

    SparseTable(vector<ll>& arr) {
        int n = arr.size();
        table = vector<vector<ll>>(n + 1, vector<ll>(20));
        lgs = vector<int>(n + 1);

        for (int i = 0; i < n; i++)
            table[i][0] = arr[i];
        for (int i = 2; i <= n; i++)
            lgs[i] = lgs[i / 2] + 1;
        for (int lg = 1; lg <= lgs[n]; lg++)
            for (int i = 0; i + (1ll << lg) - 1 < n; i++)
                table[i][lg] =
                    myFunction(table[i][lg - 1], table[i + (1ll << (lg - 1))][lg - 1]);
    }

    ll get(int l, int r) {
        return myFunction(table[l][lgs[r - l + 1]],
                          table[r - (1 << lgs[r - l + 1]) + 1][lgs[r - l + 1]]);
    }
};
```

6.10 2D Sparse Table

```
int g[N][N];
struct SparseTable2D
{
    int lg2[N];
    int st[N][N][LG][LG];
    void pre(int n, int m)
    {
        for (int i = 2; i < N; i++)
            lg2[i] = lg2[i >> 1] + 1;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                st[i][j][0][0] = g[i][j];
            }
        }
    }
    void build(int n, int m)
    {
        pre(n, m);
        for (int a = 0; a < LG; a++)
        {
            for (int b = 0; b < LG; b++)
            {
                if (a + b == 0)
                    continue;
                for (int i = 0; i + (1 << a) <= n; i++)
                {
                    for (int j = 0; j + (1 << b) <= m; j++)
                    {
                        if (!a)
                            st[i][j][a][b] = max(st[i][j][a][b - 1],

                                                 st[i][j + (1 << (b - 1))][a][b - 1]);

                        else
                            st[i][j][a][b] = max(st[i][j][a - 1][b], st[i

                                                                        + (1 << (a - 1))][j][a
    - 1][b]);
                    }
                }
            }
        }
    }
    int query(int x1, int y1, int x2, int y2)
    {
        x2++, y2++;
        int a = lg2[x2 - x1], b = lg2[y2 - y1];

        return max(
            max(st[x1][y1][a][b], st[x2 - (1 << a)][y1][a][b]),
            max(st[x1][y2 - (1 << b)][a][b], st[x2 - (1 << a)][y2 -

                                                               (1 << b)][a][b]));
    }
};
```

6.11 Sqrt Decomposition

```
const int sqt = 450;
int n{0}, q{0};
int lazy[sqt + 1]{};
int lazySet[sqt + 1]{};
int buckets[sqt + 1]{};
int arr[N]{};
void build(int buc_num)
{
    buckets[buc_num] = 0;
    for (int i = buc_num * sqt; i < min(n, (buc_num + 1) * sqt); i++)
    {
        if (lazySet[buc_num])
            arr[i] = lazySet[buc_num];
        if (lazy[buc_num])
            arr[i] += lazy[buc_num];
        buckets[buc_num] += arr[i];
    }
    lazySet[buc_num] = lazy[buc_num] = 0;
}
void updateSet(int l, int r, int x)
{
    build(l / sqt);
    build(r / sqt);
    for (int i = l; i <= r;)
    {
        if (i % sqt == 0 and i + sqt - 1 <= r)
        {
            buckets[i / sqt] = sqt * x;
            lazy[i / sqt] = 0;
```

```
            lazySet[i / sqt] = x;
            i += sqt;
        }
        else
        {
            buckets[i / sqt] -= arr[i];
            arr[i] = x;
            buckets[i / sqt] += arr[i];
            i++;
        }
    }
};
void update(int l, int r, int x)
{
    build(l / sqt);
    build(r / sqt);

    for (int i = l; i <= r;)
    {
        if (i % sqt == 0 and i + sqt - 1 <= r)
        {
            buckets[i / sqt] += sqt * x;
            lazy[i / sqt] += x;
            i += sqt;
        }
        else
        {
            buckets[i / sqt] -= arr[i];
            arr[i] += x;
            buckets[i / sqt] += arr[i];
            i++;
        }
    }
};
int query(int l, int r)
{
    build(l / sqt);
    build(r / sqt);
    int ret = 0;
    for (int i = l; i <= r;)
    {
        if (i % sqt == 0 and i + sqt - 1 <= r)
        {
            ret += buckets[i / sqt];
            i += sqt;
        }
        else
        {
            ret += arr[i];
            i++;
        }
    }
    return ret;
}
```

6.12 DSU

```
struct DSU
{
    vector<int> par, sz;
    DSU(int n)
    {
        sz = par = vector<int>(n + 1, 1);
        iota(par.begin(), par.end(), 0);
    }
    void merge(int u, int v)
    {
        u = getPar(u);
        v = getPar(v);
        if (u == v)
            return;
        if (sz[v] > sz[u])
            swap(v, u);
        par[v] = u;
        sz[u] += sz[v];
    }
    int getPar(int u)
    {
        if (par[u] == u)
            return u;
        return par[u] = getPar(par[u]);
    }
    bool get(int u, int v)
    {
        return (getPar(u) == getPar(v));
    }
};
```

6.13 DSU with Rollback

```
struct DSU
{
    vector<int> par, sz;
    stack<pair<int &, int>> rollbacks;
    DSU(int n)
    {
        par = sz = vector<int>(n + 1, 1);
        iota(all(par), 0);
    }
    int getPar(int u)
    {
        if (par[u] == u)
            return u;
        return getPar(par[u]); // No path compression due to rollbacks
    }
    void merge(int u, int v)
    {
        u = getPar(u);
        v = getPar(v);
        rollbacks.push({par[u], par[u]});
        rollbacks.push({par[v], par[v]});
        rollbacks.push({sz[u], sz[u]});
        rollbacks.push({sz[v], sz[v]});
        if (u == v)
            return;
        if (sz[v] > sz[u])
            swap(v, u);
        sz[u] += sz[v];
```

```
        par[v] = u;
    }
    void rollback()
    {
        assert(rollbacks.size() >= 4);
        for (int i = 0; i < 4; i++)
        {
            auto [variable, value] = rollbacks.top();
            variable = value;
            rollbacks.pop();
        }
    }
    int get(int u, int v)
    {
        return getPar(u) == getPar(v);
    }
};
```

6.14 Mo's Algorithm

```
const int sqt = 320;
struct query
{
    int l, r, i;
    bool operator<(query &other)
    {
        int b1 = l / sqt;
        int b2 = other.l / sqt;
        if(b1 != b2) return b1 < b2;
        return r < other.r;
    }
};

int freq[N]{};
int n, q;
int answer = 0;
int arr[N]{};
void add(int idx)
{
    int val = arr[idx];
    if(val > N)
        return;
    answer -= (freq[val] == val);
    freq[val]++;
    answer += (freq[val] == val);
}
void rem(int idx)
{
    int val = arr[idx];
    if(val > N)
        return;

    answer -= (freq[val] == val);
    freq[val]--;
    answer += (freq[val] == val);
}

void MosAlgo(vector<query> &queries, vector<int> &ans)
{
    sort(all(queries));
    int l = 0, r = -1;
    for(auto &[L, R, idx] : queries)
    {
        while (r < R) add(++r);
        while(l > L) add(--l);
        while(r > R) rem(r--);
        while(l < L) rem(l++);
        ans[idx] = answer;
    }
}
```

6.15 Mo on Hilbert Order

```
inline int64_t hilbertOrder(int x, int y, int pow, int rotate)
{
    if (pow == 0)
    {
        return 0;
    }
    int hpow = 1 << (pow - 1);
    int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 : 2);
    seg = (seg + rotate) & 3;
    const int rotateDelta[4] = {3, 0, 0, 1};
    int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
    int nrot = (rotate + rotateDelta[seg]) & 3;
    int64_t subSquareSize = int64_t(1) << (2 * pow - 2);
    int64_t ans = seg * subSquareSize;
    int64_t add = hilbertOrder(nx, ny, pow - 1, nrot);
    ans += (seg == 1 || seg == 2) ? add : (subSquareSize - add - 1);
    return ans;
}
```

6.16 Monotonic Stack & Queue

```
template <class T>
struct Mono_stack
{
    stack<pair<T, T>> st;
    void push(const T &val)
    {
        if (st.empty())
            st.emplace(val, val);
        else
            st.emplace(val, std::max(val, st.top().second));
    }
    void pop()
    {
        st.pop();
    }
    bool empty()
    {
        return st.empty();
    }
    int size()
    {
        return st.size();
    }
    T top()
```

```
    {
        return st.top().first;
    }
    T max()
    {
        return st.top().second;
    }
};
template <class T>
struct Mono_queue
{
    Mono_stack<T> pop_st, push_st;
    void push(const T &val)
    {
        push_st.push(val);
    }
    void move()
    {
        if (pop_st.size())
            return;
        while (!push_st.empty())
            pop_st.push(push_st.top()), push_st.pop();
    }
    void pop()
    {
        move();
        pop_st.pop();
    }
    bool empty()
    {
        return pop_st.empty() && push_st.empty();
    }
    int size()
    {
        return pop_st.size() + push_st.size();
    }
    T top()
    {
        move();
        return pop_st.top();
    }
    T max()
    {
        if (pop_st.empty())
            return push_st.max();
        if (push_st.empty())
            return pop_st.max();
        return std::max(push_st.max(), pop_st.max());
    }
};
```

6.17 2-SAT

```
int n, m;
vector<int> a[N];
int dfsn[N], lowLink[N], inStack[N], comp[N], repr[N], dtime, scc;
stack<int> st;
void tarjan(int node)
{
    dfsn[node] = lowLink[node] = dtime++, inStack[node] = 1;
    st.push(node);
    for (auto &i : a[node])
    {
        if (dfsn[i] == -1)
        {
            tarjan(i);
            lowLink[node] = min(lowLink[node], lowLink[i]);
        }
        else if (inStack[i])
            lowLink[node] = min(lowLink[node], dfsn[i]);
    }
    if (dfsn[node] == lowLink[node])
    {
        int x = -1;
        while (x != node)
        {
            x = st.top(), st.pop();
            inStack[x] = 0;
            comp[x] = scc;
        }
        repr[scc] = node;
        scc++;
    }
}
int NOT(int x)
{
    return 2 * m - x + 1;
}
void add(int x, int y)
{
    a[NOT(x)].push_back(y);
    a[NOT(y)].push_back(x);
}
void doWork()
{
    cin >> n >> m;
    for (int i = 0; i < n; i++)
    {
        char op;
        int x, y;
        cin >> op >> x;
        if (op == '-')
            x = NOT(x);
        cin >> op >> y;
        if (op == '-')
            y = NOT(y);
        add(x, y);
    }
    memset(dfsn, -1, sizeof dfsn);
    for (int i = 1; i <= 2 * m; i++)
    {
        if (dfsn[i] == -1)
            tarjan(i);
    }
    for (int i = 1; i <= m; i++)
    {
        if (comp[i] == comp[NOT(i)])
        {
```

```
            cout << "IMPOSSIBLE";
            return;
        }
    }
    vector<int> comp_values(scc + 1, -1), ans(m + 1);
    for (int i = 0; i < scc; i++)
    {
        if (comp_values[i] == -1)
        {
            comp_values[i] = 1;
            int dual = comp[NOT(repr[i])];
            comp_values[dual] = 0;
        }
    }
    for (int i = 1; i <= m; i++)
        ans[i] = comp_values[comp[i]];
}
```

6.18 Wavelet Tree

```
// Wavelet Tree — O(n log σ) build, O(log σ) query
// Positions 1-indexed [L,R]. Build: WT(arr+1, arr+n+1, lo, hi)
//
// Derivable (not in code):
//   lt(L,R,v)            = lte(L,R,v-1)
//   range_count(L,R,a,b) = lte(L,R,b) - lte(L,R,a-1)
//   range_sum(L,R,a,b)   = sum_lte(L,R,b) - sum_lte(L,R,a-1)
//   sum_k(L,R,k)         = sum_lte(L,R, kth(L,R,k))
//                            - kth(L,R,k) * (lte(L,R,kth(L,R,k)) - k)
//   prev_value(L,R,v)    = kth(L,R, lte(L,R,v))       — -1 if lte==0
//   next_value(L,R,v)    = kth(L,R, lte(L,R,v-1)+1)   — -1 if lte==R-L+1
struct WT {
    int lo, hi;
    vector<int> b;
    vector<ll> t; // b[i]=prefix count left, t[i]=prefix sum left
    WT *l = NULL, *r = NULL;

    WT(int *from, int *to, int x, int y) {
        lo = x, hi = y;
        if (lo == hi || from == to)
            return;
        int mid = lo + (hi - lo) / 2;
        int n = to - from;
        b.assign(n + 1, 0);
        t.assign(n + 1, 0);
        for (int i = 0; i < n; i++) {
            b[i + 1] = b[i] + (from[i] <= mid);
            t[i + 1] = t[i] + (from[i] <= mid ? from[i] : 0);
        }
        auto f = [mid](int v) { return v <= mid; };
        auto pivot = stable_partition(from, to, f);
        if (from != pivot)
            l = new WT(from, pivot, lo, mid);
        if (pivot != to)
            r = new WT(pivot, to, mid + 1, hi);
    }

    // # values <= k in [L,R]
    int lte(int L, int R, int k) {
        if (L > R || k < lo)
            return 0;
        if (hi <= k)
            return R - L + 1;
        int lb = b[L - 1], rb = b[R];
        int mid = lo + (hi - lo) / 2;
        if (k <= mid) {
            if (l)
                return l->lte(lb + 1, rb, k);
            return 0;
        }
        int left_count = rb - lb;
        if (r)
            return left_count + r->lte(L - lb, R - rb, k);
        return left_count;
    }

    // k-th smallest in [L,R], 1-indexed
    int kth(int L, int R, int k) {
        if (lo == hi)
            return lo;
        int lb = b[L - 1], rb = b[R], inL = rb - lb;
        if (k <= inL)
            return l->kth(lb + 1, rb, k);
        return r->kth(L - lb, R - rb, k - inL);
    }

    // sum of values <= v in [L,R]
    ll sum_lte(int L, int R, int v) {
        if (L > R || v < lo)
            return 0;

        // Base case: leaf node
        if (lo == hi) {
            return 1LL * (R - L + 1) * lo;
        }

        int lb = b[L - 1], rb = b[R];
        int mid = lo + (hi - lo) / 2;

        // Traverse left
        if (v <= mid) {
            if (l)
                return l->sum_lte(lb + 1, rb, v);
            return 0;
        }

        // Traverse right (this naturally handles the hi <= v case)
        ll left_sum = t[R] - t[L - 1];
        if (r)
            return left_sum + r->sum_lte(L - lb, R - rb, v);
        return left_sum;
    }

    // # distinct values in [L,R]
    int count_distinct(int L, int R) {
        if (L > R)
            return 0;
        if (lo == hi)
```

```
            return 1;
        int lb = b[L - 1], rb = b[R];
        int res = 0;
        if (lb < rb)
            res += l->count_distinct(lb + 1, rb);
        if (L - lb <= R - rb)
            res += r->count_distinct(L - lb, R - rb);
        return res;
    }

    ~WT() {
        delete l;
        delete r;
    }
};
```

6.19 Xor Basis

```
struct XB
{
    static constexpr int B = 60;
    array<ll, B + 1> base{};
    int rank = 0;
    XB(int val = 0)
    {
        if (val)
            insert(val);
    }
    void insert(ll x)
    {
        for (int i = B; i >= 0; --i)
        {
            if (((x >> i) & 1LL) == 0)
                continue;
            if (base[i] == 0)
            {
                base[i] = x;
                ++rank;
                return;
            }
            x ^= base[i];
        }
    }

    // Check if x can be formed as XOR of some inserted numbers
    bool canRepresent(ll x) const
    {
        for (int i = B; i >= 0; --i)
        {
            if (((x >> i) & 1LL) && base[i])
                x ^= base[i];
        }
        return x == 0;
    }

    // Maximum XOR value achievable from any subset (starting from 0 by default)
    ll getMax(ll start = 0) const
    {
        ll res = start;
        for (int i = B; i >= 0; --i)
        {
            if (!base[i])
                continue;
            res = max(res, res ^ base[i]);
        }
        return res;
    }
    ll getKth(ll k)
    {
        if(k > (1ll << rank))
            return -1;
        k--;
        ll res = 0;
        for (int i = B, j = rank - 1; i >= 0; --i)
        {
            if (!base[i])
                continue;
            if ((k >> j) & 1)
                res = max(res, res ^ base[i]);
            else
                res = min(res, res ^ base[i]);
            j--;
        }
        return res;
    }

    int getRank() const
    {
        return rank;
    }
    const XB operator+(const XB &other) const
    {
        XB ret = *this;
        for (int i = B; i >= 0; --i)
        {
            if (other.base[i])
                ret.insert(other.base[i]);
        }
        return ret;
    }
};


// Xor Basis in Range (Offline precompute)


constexpr int B = 60;

ll basis[B + 1];
int closest[B + 1];
void insert(ll x, int idx)
{
    for (int i = B; i >= 0; i--)
    {
        if (!(x >> i & 1))
            continue;
        if (closest[i] < idx)
        {
```

```
            swap(x, basis[i]);
            swap(closest[i], idx);
        }
        x ^= basis[i];
    }
}
bool canRepresent(int l, ll x)
{
    for (int i = B; i >= 0; i--)
    {
        if (!(x >> i & 1))
            continue;
        if (closest[i] < l)
            return false;
        x ^= basis[i];
    }
    return true;
}

// Xor Basis with Bitset

constexpr int B = 1504;

using big = bitset<B + 1>;
big basis[B + 1];
int ids[B + 1];
big msk[B + 1];
const big one = 1;
bool insert(big x, int id)
{
    big mask = 0;
    for (int i = B; i >= 0; i--)
    {
        if (!x[i])
            continue;
        if (basis[i] == 0)
        {
            basis[i] = x;
            ids[i] = id;
            msk[i] = mask ^ (one << i);
            return true;
        }
        mask ^= msk[i];
        x ^= basis[i];
    }
    return false;
}
int n, k;
vector<int> can(big x)
{
    big mask = 0;
    for (int i = B; i >= 0; i--)
    {
        if (x[i] and basis[i] != 0)
            x ^= basis[i], mask ^= msk[i];
    }
    if (x != 0 or mask.count() == n)
        return {-1};
    vector<int> ret;
    for (int i = B; i >= 0; i--)
        if (mask[i])
            ret.push_back(ids[i]);
    return ret;
}
```

7 Graphs

7.1 Graph Template

```
bool visited[N]{};
void dfs(vector<vector<int>> &graph, int st)
{
    visited[st] = true;
    for (auto &child : graph[st])
        if (!visited[child])
            dfs(graph, child);
}

vector<int> bfs(vector<vector<int>> &graph, int st)
{
    vector<int> dist(graph.size(), INF);
    queue<int> nextToVisit;
    dist[st] = 0;
    nextToVisit.push(st);
    while (!nextToVisit.empty())
    {
        auto cur = nextToVisit.front();
        nextToVisit.pop();
        for (auto &ch : graph[cur])
        {
            if (dist[ch] == INF)
            {
                dist[ch] = dist[cur] + 1;
                nextToVisit.push(ch);
            }
        }
    }
    return dist;
}

vector<int> dijk(vector<vector<pii>> &graph, int st)
{
    vector<int> dist(graph.size(), INF);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>
     nextToVisit;
    dist[st] = 0;
    nextToVisit.push({0, st});
    while (nextToVisit.size())
    {
        auto [curCost, curDist] = nextToVisit.top();
        nextToVisit.pop();
        if (curCost > dist[curDist])
            continue;
        for (auto &[cw, cd] : graph[curDist])
        {
            if (dist[cd] > cw + curCost)
            {
```

```
                dist[cd] = cw + curCost;
                nextToVisit.push({dist[cd], cd});
            }
        }
    }
    return dist;
}

void kahn(vector<vector<int>> &graph, int &n)
{
    vector<int> degree(n + 1, 0);
    vector<bool> visited(n + 1, false);
    for (int i = 1; i <= n; i++)
    {
        for (auto &j : graph[i])
        {
            degree[j]++;
        }
    }
    queue<int> nextToVisit;
    for (int i = 1; i <= n; i++)
    {
        if (!degree[i])
        {
            nextToVisit.push(i);
            visited[i] = true;
        }
    }
    vector<int> result;
    while (nextToVisit.size())
    {
        int cur = nextToVisit.front();
        nextToVisit.pop();
        result.push_back(cur);
        for (auto &j : graph[cur])
        {
            degree[j]--;
            if (!degree[j])
                nextToVisit.push(j);
        }
    }
    if (result.size() != n)
    {
        cout << "IMPOSSIBLE" << endl;
    }
    else
    {
        for (auto &j : result)
            cout << j << ' ';
        cout << endl;
    }
}

vector<int> BellmanFord(vector<vector<pii>> &graph, int &V, int st)
{
    vector<int> dist(V + 1, INF);
    dist[st] = 0;
    bool flag = false;
    for (int edgesUsed = 1; edgesUsed <= V; edgesUsed++)
    {
        for (int u = 1; u <= V; u++)
        {
            if (dist[u] == INF)
                continue;
            for (auto &[cost, v] : graph[u])
            {
                if(dist[v] > dist[u] + cost)
                    dist[v] = dist[u] + cost, flag |= (v == V and edgesUsed == V);
            }
        }
    }
    if(flag)
        return {};
    return dist;
}
void floyd()
{
    int d[N][N];
    for(int k = 1; k <= n; k++)
        for(int u = 1; u <= n; u++)
            for(int v = 1; v <= n; v++)
                d[u][v] = max(d[u][v], d[u][k] + d[k][v]);
}
```

7.2 Bellman-Ford

```

struct Edge {
    int u, v;
    long long w;
};

void solve() {
    int n , m;
    cin >> n >> m;
    vector<Edge> edges(m);
    for (int i = 0; i < m; ++i)
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    // source 0 linked with each nodes
    vector<ll> dist(n + 1, 0);

    for (int i = 1; i < n; ++i) {
        bool changed = false;
        for (const auto& e : edges) {
            if (dist[e.v] > dist[e.u] + e.w) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        }
        if (!changed)
            break;
    }

    for (Edge e : edges)
        if (dist[e.v] > dist[e.u] + e.w)
            return void(cout << "-inf\n");

    ll min_dist = 1e18;
```

```
    for (int i = 1; i <= n; ++i)
        min_dist = min(min_dist, dist[i]);

    cout << min_dist << endl;
}
```

7.3 SPFA

```
const ll OO = 1e15;
const int N = 2500 + 5;
vector<pair<int, ll>> adj[N];
// st and par are optional, just for finding negative
cycles
    vi st;
int n;
bool SPFA(int src, vector<ll> &d)
{
    fill(d.begin(), d.end(), OO);
    vi cnt(n + 1), par(n + 1, -1);
    vector<bool> inQ(n + 1, false);
    queue<int> q;
    d[src] = 0;
    q.push(src);
    inQ[src] = true;
    while (!q.empty())
    {
        int p = q.front();
        q.pop();
        inQ[p] = false;
        for (auto e : adj[p])
        {
            int to = e.F;
            ll w = e.S;
            if (d[p] + w < d[to])
            {
                d[to] = max(-OO, d[p] + w);
                par[to] = p;
                if (!inQ[to])
                {
                    inQ[to] = true;
                    if (++cnt[to] > n)
                        st.pb(to);
                    else
                        q.push(to);
                }
            }
        }
    }
    sort(st.begin(), st.end());
    st.erase(unique(st.begin(), st.end()), st.end());
    for (auto &e : st)
        for (int i = 0; i < n; i++)
            e = par[e];
    return st.empty();
}
```

7.4 MST (Kruskal)

```
DSU dsu(n);
vector<edge> edges;
for (int i = 0; i < m; i++)
{
    int c, u, v;
    cin >> u >> v >> c;
    edges.push_back({c, u, v});
}
sort(all(edges));
int sum = 0;
for (auto &[c, u, v] : edges)
{
    if (dsu.get(u, v))
        continue;
    sum += c;
    dsu.merge(u, v);
}
```

7.5 Second Best MST

```
int main()
{
    int n, m;
    cin >> n >> m;
    vector<edge> edges;
    for (int i = 0, u, v, w; i < m; i++)
        cin >> u >> v >> w, edges.emplace_back(edge(u, v, w, i));
    dsu.init(n);
    int mst = kruskal(edges); // also fills the graph with the mst
    up[1][0] = 1;
    mx[1][0] = INT_MIN;
    dfs(1, 1);
    int lca, val, u, v;
    int secondMst = INT_MAX;
    for (auto &edge : edges) // try all edges
    {
        if (usedInMst[edge.id])
            continue;
        u = edge.u, v = edge.v;
        lca = LCA(u, v);
        val = mst - max(getMax(edge.u, lca), getMax(edge.v, lca)) +
              edge.w;
        if (val > mst)
            secondMst = min(secondMst, val);
    }
    cout << secondMst;
}
```

7.6 LCA

```
const int N = 1e5 + 5, LG = 23;
vector<int> graph[N];
int up[LG][N];
int deep[N]{};
void dfs(int cur, int par = -1, int depth = 0)
{
    deep[cur] = depth;
    for(auto &ch : graph[cur])
    {
        if(ch == par)
```

```
            continue;
        up[0][ch] = cur;
        for(int lg = 1; lg < LG and ~up[lg - 1][ch]; lg++)
            up[lg][ch] = up[lg - 1][up[lg - 1][ch]];
        dfs(ch, cur, depth +1);
    }
}
int kthAnc(int u, int k)
{
    for(int lg = 0; lg < LG and ~u; lg++)
        if((k >> lg) & 1)
            u = up[lg][u];
    return u;
}
int getLca(int u, int v)
{
    if(deep[v] > deep[u])
        swap(v, u);
    u = kthAnc(u, deep[u] - deep[v]);
    if(u == v)
        return u;
    for(int lg = LG - 1; lg >= 0; --lg)
        if(up[lg][u] != up[lg][v])
            u = up[lg][u], v = up[lg][v];
    return up[0][u];
}
```

7.7 LCA with Sparse

```
template <typename T>
struct SparseTable
{
    vector<vector<T>> table;
    vector<int> lgs;
    function<T(T, T)> myFunction;

    void build(T arr[], function<T(T, T)> fun, int n)
    {
        myFunction = fun;
        table = vector<vector<T>>(n + 1, vector<T>(20));
        lgs = vector<T>(n + 1);
        for (int i = 0; i < n; i++)
            table[i][0] = arr[i];
        for (int i = 2; i <= n; i++)
            lgs[i] = lgs[i / 2] + 1;
        for (int lg = 1; lg <= lgs[n]; lg++)
            for (int i = 0; i + (1 << lg) - 1 < n; i++)
                table[i][lg] = myFunction(table[i][lg - 1], table[i + (1 << (lg - 1))][lg -
 1]);
    }
    T get(int l, int r)
    {
        return myFunction(table[l][lgs[r - l + 1]], table[r - (1 << lgs[r - l + 1]) + 1][lgs[r
  - l + 1]]);
    }
};
SparseTable<int> sp;
vector<int> graph[N];
int first[N];
int eular[2 * N];
int deep[N];
int timer = 0;
void eular_tour(int cur, int par = -1, int depth = 0)
{
    deep[cur] = depth;
    eular[timer] = cur;
    first[cur] = timer++;
    for (auto &ch : graph[cur])
        if (ch != par)
            eular_tour(ch, cur, depth + 1), eular[timer++] = cur;
}
#define getLca(x, y) sp.get(min(first[x], first[y]), max(first[x], first[y]))
#define getDist(x, y) deep[x] + deep[y] - ((deep[getLca(x, y)]) << 1)
```

7.8 Fast LCA

```
vector<int> g[N];
vector<pair<int, int>> queries[N]; // queries[u] contains {v, idx}
int ancestor[N], lca_answer[N];
bool visited[N];

void dfs(int u, int p, DSU &dsu) {
    ancestor[u] = u;
    visited[u] = true;

    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u, dsu);
        dsu.merge(u, v);
        ancestor[dsu.getPar(u)] = u;
    }

    for (auto [v, idx] : queries[u]) {
        if (visited[v]) {
            lca_answer[idx] = ancestor[dsu.getPar(v)];
        }
    }
}
```

7.9 HLD

```
struct HLD
{
    int n, timer;
    vector<int> flat, sz, tp, dep, par;
    vector<vector<int>> graph;
    FenwickTree bit;
    HLD(vector<vector<int>> &graph) : graph(graph), n(graph.size() - 1)
    {
        timer = 0;
        sz = flat = tp = dep = par = vector<int>(n + 1, 0);
        preDfs(1);
        dfs(1);
        bit.resize(n + 1);
        for(int i = 1; i <= n;i++)
            update(i, 1);
    }
    void preDfs(int cur)
```

```
    {
        sz[cur] = 1;
        for (auto &ch : graph[cur])
        {
            graph[ch].erase(find(all(graph[ch]), cur));
            dep[ch] = dep[cur] + 1;
            par[ch] = cur;
            preDfs(ch);
            sz[cur] += sz[ch];
        }
        auto __cmp = [&](int u, int v) { return (sz[u] < sz[v]); };
        int heavy = max_element(all(graph[cur]), __cmp) - graph[cur].begin();
        if (graph[cur].size())
        {
            swap(graph[cur][0], graph[cur][heavy]);
        }
    }
    void dfs(int cur, int top = 1)
    {
        flat[cur] = ++timer, tp[cur] = top;
        for (int i = 0; i < graph[cur].size(); i++)
        {
            auto &ch = graph[cur][i];
            dfs(ch, !i ? top : ch);
        }
    }
    void update(int cur, int val)
    {
        bit.update(flat[cur], val);
    }
    int query(int u, int v)
    {
        int ret = 0;
        for (; tp[u] != tp[v]; u = par[tp[u]])
        {
            if (dep[tp[u]] < dep[tp[v]])
                swap(u, v);
            ret += bit.get(flat[tp[u]], flat[u]);
        }
        if (dep[u] < dep[v])
            swap(u, v);
        ret += bit.get(flat[v], flat[u]);
        return ret;
    }
};
```

7.10 Centroid Decomposition

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
using ll = long long;
const int N = 2e5 + 3;
vector<int> g[N];
int sz[N];
int cnt[N];
bool isRem[N]{};
int n, k;
ll ans = 0;

void dfsSz(int u, int p)
{
    sz[u] = 1;
    for (auto &v : g[u])
        if (v != p and !isRem[v])
            dfsSz(v, u), sz[u] += sz[v];
}

int getCent(int u, int p, int cur_sz)
{
    for (auto &v : g[u])
        if (v != p and !isRem[v])
            if (2 * sz[v] > cur_sz)
                return getCent(v, u, cur_sz);
    return u;
}

void dfsAdd(int u, int p, int depth)
{
    cnt[depth]++;
    for (auto &v : g[u])
        if (v != p and !isRem[v])
            dfsAdd(v, u, depth + 1);
}
void getAns(int u, int p, int depth)
{
    if (depth <= k)
        ans += cnt[k - depth];
    for (auto &v : g[u])
        if (v != p and !isRem[v])
            getAns(v, u, depth + 1);
}
void decompose(int u)
{
    dfsSz(u, -1);
    int cent = getCent(u, -1, sz[u]);

    // Logic For Answer Here
    cnt[0] = 1;
    for (auto &v : g[cent])
        if (!isRem[v])
            getAns(v, cent, 1), dfsAdd(v, cent, 1);
    // clear
    memset(cnt, 0, sz[u] * (sizeof cnt[0]));

    // Solving for other subtrees
    isRem[cent] = 1;
    for (auto &v : g[cent])
        if (!isRem[v])
            decompose(v);
}

void solve()
{
    cin >> n >> k;
    int m = n - 1;
    while (m--)
    {
        int u, v;
```

```
        cin >> u >> v, u--, v--;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    decompose(0);
    cout << ans << endl;
}

signed main()
{
    cin.tie(0);
    ios_base::sync_with_stdio(0);
    int t{1};
    // cin >> t;
    while (t--)
        solve();
}
```

7.11 DSU on Tree (SACK)

```
int c[N];
int n;
vector<int> graph[N];
int heavy[N];
int freq[N];
int subtree[N];
int answer = 0;
int ans[N];
void update(int val, int delta)
{
    answer -= (freq[val] > 0);
    freq[val] += delta;
    answer += (freq[val] > 0);
}
void compress()
{
    map<int, int> mp;
    for (int i = 1; i <= n; i++)
        if (!mp.count(c[i]))
            mp[c[i]] = mp.size() + 1;
    for (int i = 1; i <= n; i++)
        c[i] = mp[c[i]];
}
void dfs0(int cur, int par)
{
    for (auto &ch : graph[cur])
    {
        if (ch == par)
            continue;
        dfs0(ch, cur);
        if (subtree[heavy[cur]] < subtree[ch])
            heavy[cur] = ch;
        subtree[cur] += subtree[ch];
    }
    subtree[cur]++;
}
void calc(int cur, int par, int delta)
{
    update(c[cur], delta);
    for (auto &ch : graph[cur])
        if (ch != par)
            calc(ch, cur, delta);
}
void dfs(int cur, int par, int keep)
{
    for (auto &ch : graph[cur])
    {
        if (ch == par || ch == heavy[cur])
            continue;
        dfs(ch, cur, 0);
    }
    if (heavy[cur])
        dfs(heavy[cur], cur, 1);
    for (auto &ch : graph[cur])
    {
        if (ch == par || ch == heavy[cur])
            continue;
        calc(ch, cur, 1);
    }
    update(c[cur], 1);
    ans[cur] = answer;
    if (!keep)
        calc(cur, par, -1);
}
```

7.12 Dsu on Tree

```
int c[N];
vector<int> graph[N];
set<int> s[N];
int ans[N];
void dfs(int cur,int par)
{
    s[cur].insert(c[cur]);
    for(auto &ch : graph[cur])
    {
        if(ch == par)
            continue;
        dfs(ch, cur);
        if(s[ch].size() > s[cur].size())
            swap(s[ch], s[cur]);
        for(auto &x : s[ch])
            s[cur].insert(x);
    }
    ans[cur] = s[cur].size();
}
```

7.13 Knapsack on Tree

```
vector <int> adj[N];
ll dp[N][N], val[N], sub[N];
int n;
void dfs(int node, int p)
{
    sub[node] = 1;
    vector <ll> curDP(n+5, -1e18), prvDP(n+5, -1e18);
    prvDP[0] = 0;
    prvDP[1] = val[node];
```

```
    for (auto &child : adj[node]) {
        if (child == p)
            continue;
        dfs(child, node);

        for (int sz1 = 1; sz1 <= sub[node]; sz1++)
            for (int sz2 = 0; sz2 <= sub[child]; sz2++)
                curDP[sz1+sz2] = max(curDP[sz1+sz2], prvDP[sz1] + dp[child][sz2]);
        curDP[0] = 0;
        sub[node] += sub[child];
        prvDP = curDP;
    }

    for (int i = 0; i <= sub[node]; i++)
        dp[node][i] = prvDP[i];
}
```

7.14 Block Cut Tree

```
const int N = 4e5 + 3, LG = 23;

// ---- Tarjan & Block Cut Tree Part ----
vector<int> graph[N];
vector<int> BCT[N];
int isArt[N];
int low[N];
int dfsn[N];
int n, timer = 0, m, q;
int inBCT[N], valInBCT[N];

stack<int> st;
vector<vector<int>> comps;
bool rootEdge = false;
void tarjan(int cur = 1, int par = -1)
{
    low[cur] = dfsn[cur] = timer++;
    st.push(cur);
    for (auto &ch : graph[cur])
    {
        if (dfsn[ch] == -1)
        {
            tarjan(ch, cur);
            low[cur] = min(low[cur], low[ch]);
            if (low[ch] >= dfsn[cur])
            {
                if (dfsn[cur] == 0 and !rootEdge)
                    rootEdge = true;
                else
                    isArt[cur] = true;
                comps.push_back({cur});
                while (comps.back().back() != ch)
                    comps.back().push_back(st.top()), st.pop();
            }
        }
        else if (ch != par)
        {
            low[cur] = min(low[cur], dfsn[ch]);
        }
    }
}

inline void addEdgeTree(int u, int v)
{
    BCT[u].push_back(v), BCT[v].push_back(u);
}

inline void buildBCT()
{
    timer = 0;
    for (int i = 1; i <= n; i++)
        if (isArt[i])
            inBCT[i] = ++timer, valInBCT[inBCT[i]] = 1;
    for (auto &comp : comps)
    {
        int id = ++timer;
        for (auto &cur : comp)
            (isArt[cur]) ? (addEdgeTree(id, inBCT[cur])) : (inBCT[cur] = id, void());
    }
}

// ---- LCA  Part ----
int up[N][LG];
int deep[N];
int deepVal[N];
void dfs(int cur, int par)
{
    for (auto &ch : BCT[cur])
    {
        if (ch == par)
            continue;
        up[ch][0] = cur;
        deep[ch] = deep[cur] + 1;
        deepVal[ch] = deepVal[cur] + valInBCT[ch];
        for (int lg = 1; lg < LG and ~up[ch][lg - 1]; lg++)
            up[ch][lg] = up[up[ch][lg - 1]][lg - 1];
        dfs(ch, cur);
    }
}
inline int kthAnc(int u, int k)
{
    for (int lg = LG - 1; lg >= 0 and ~u; lg--)
        if ((k >> lg) & 1)
            u = up[u][lg];
    return u;
}
inline int getLca(int u, int v)
{
    if (deep[v] > deep[u])
        swap(v, u);
    u = kthAnc(u, deep[u] - deep[v]);
    if (u == v)
        return u;
    for (int lg = LG - 1; lg >= 0; lg--)
        if (up[u][lg] != up[v][lg])
            u = up[u][lg], v = up[v][lg];
    return up[u][0];
}
```

7.15 Tarjan Articulation Points

```cpp
int tarjan(vector<vector<int>> &graph)
{
    int n = graph.size() - 1;
    vector<int> dfsn(n + 1, -1);
    vector<int> low(n + 1, -1);
    vector<int> isArt(n + 1, 0);
    int timer = 0;
    function<void(int, int)> dfs = [&](int cur, int par) {
        low[cur] = dfsn[cur] = timer++;
        bool rootEdge = false;
        for (auto &ch : graph[cur])
        {
            if (dfsn[ch] == -1)
            {
                dfs(ch, cur);
                low[cur] = min(low[cur], low[ch]);
                if (low[ch] >= dfsn[cur])
                {
                    if(dfsn[cur] == 0 and !rootEdge)
                        rootEdge = true;
                    else
                        isArt[cur] = true;
                }
            }
            else if (ch != par)
            {
                low[cur] = min(low[cur], dfsn[ch]);
            }
        }
    };
    dfs(1, -1);
    vector<int> artPoints;
    for (int i = 1; i <= n; i++)
        if (isArt[i])
            artPoints.push_back(i);
    return artPoints.size();
}
```

7.16 Tarjan Bridge Tree

```cpp
int tarjan(vector<vector<int>> &graph)
{
    int n = graph.size() - 1;
    vector<int> dfsn(n + 1, -1), low(n + 1, -1);
    int ret = 0;
    int timer = 0;
    set<pii> bridges;
    function<void(int, int)> dfs = [&](int cur, int par) {
        dfsn[cur] = low[cur] = timer++;
        for (auto &ch : graph[cur])
        {
            if (dfsn[ch] == -1)
            {
                dfs(ch, cur);
                low[cur] = min(low[cur], low[ch]);
                if (low[ch] == dfsn[ch])
                    bridges.insert({cur, ch});
            }
            else if (ch != par)
                low[cur] = min(low[cur], dfsn[ch]);
        }
    };
    dfs(1, -1);
    vector<int> comp(n + 1, -1);
    vector<vector<int>> bridgesTree(n + 1);
    timer = 1;
    function<void(int)> calcComp = [&](int cur) {
        for (auto &ch : graph[cur])
            if (comp[ch] == -1 and !bridges.count({cur, ch}) and !bridges.count({ch, cur}))
                comp[ch] = comp[cur], calcComp(ch);
    };
    for (int i = 1; i <= n; i++)
        if (comp[i] == -1)
            comp[i] = timer++, calcComp(i);
    for (auto &[u, v] : bridges)
    {
        int cmpU = comp[u];
        int cmpV = comp[v];
        bridgesTree[cmpU].push_back(cmpV);
        bridgesTree[cmpV].push_back(cmpU);
    }
    int mx = -1, node = -1;
    function<void(int, int , int)> dfs2 = [&](int cur, int par, int depth)
    {
        if(depth > mx)
            node = cur, mx = depth;
        for(auto &ch : bridgesTree[cur])
            if(ch != par)
                dfs2(ch, cur, depth + 1);
    };
    dfs2(1, -1, 0);
    mx = -1;
    dfs2(node, -1, 0);
    return mx;
}
```

7.17 Tarjan SCC

```cpp
int tarjan(vector<vector<int>> &graph)
{
    int n = graph.size() - 1;
    vector<int> low(n + 1, -1), inStack(n + 1, 0), dfsn(n + 1, -1), comp(n + 1, -1);
    vector<vector<int>> comps;
    stack<int> st;
    int timer = 0;
    function<void(int)> dfs = [&](int cur) {
        low[cur] = dfsn[cur] = timer++;
        inStack[cur] = 1;
        st.push(cur);
        for (auto &ch : graph[cur])
        {
            if (dfsn[ch] == -1)
            {
                dfs(ch);
                low[cur] = min(low[cur], low[ch]);
```

```
            }
            else if (inStack[ch])
            {
                low[cur] = min(low[cur], dfsn[ch]);
            }
        }
        if (low[cur] == dfsn[cur])
        {
            comps.emplace_back(); // Add New Comp
            int node = -1;
            while (node != cur)
            {
                node = st.top();
                st.pop();
                inStack[node] = 0;
                comps.back().push_back(node);
                comp[node] = comps.back().size() - 1;
            }
        }

    };
    for (int i = 1; i <= n; i++)
        if (dfsn[i] == -1)
            dfs(i);
    return (comps.size() == 1);
}
```

7.18 Kosaraju

```
int kosaraju(vector<vector<int>> &graph, vector<vector<int>> &revGraph)
{
    stack<int> st;
    int n = graph.size() - 1;
    vector<int> visited(n + 1, 0);
    function<void(int)> dfs1 = [&](int cur)
    {
        if (visited[cur])
            return;
        visited[cur] = 1;
        for (auto &ch : graph[cur])
            dfs1(ch);
        st.push(cur);
    };
    function<void(int)> dfs2 = [&](int cur)
    {
        if (!visited[cur])
            return;
        visited[cur] = 0;
        for (auto &ch : revGraph[cur])
            dfs2(ch);
    };
    for (int i = 1; i <= n; i++)
        if (!visited[i])
            dfs1(i);
    int sccCount = 0;
    while (st.size())
    {
        int cur = st.top();
        st.pop();
        if (visited[cur])
        {
            dfs2(cur);
            sccCount++;
        }
    }
    return (sccCount == 1);
};
```

7.19 Check for Odd Cycle

```
bool CheckOddCycle()
{
    int vis[N + 2];
    memset(vis, -1, sizeof vis);
    vis[1] = 1;
    queue<int> q;
    q.push(1);
    while (!q.empty())
    {
        int node = q.front();
        q.pop();
        for (auto it : v[node])
        {
            if (vis[it] == -1)
            {
                vis[it] = 1 - vis[node];
                q.push(it);
            }
            else if (vis[it] == vis[node])
                return true;
        }
    }
    return false;
}
```

7.20 Finding Negative Cycle

```
struct Edge
{
    int a, b, cost;
};
int n, m;
vector<Edge> edges;
const int INF = 1000000000;
void solve()
{
    vector<int> d(n), p(n, -1);
    int x;
    for (int i = 0; i < n; ++i)
    {
        x = -1;
        for (Edge e : edges)
            if (d[e.a] + e.cost < d[e.b])
            {
                d[e.b] = d[e.a] + e.cost;
                p[e.b] = e.a;
                x = e.b;
            }
```

```
    }
    if (x == -1)
        cout << "No negative cycle found.";
    else
    {
        for (int i = 0; i < n; ++i)
            x = p[x];
        vector<int> cycle;
        for (int v = x;; v = p[v])
        {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1)
                break;
        }
        reverse(cycle.begin(), cycle.end());
        cout << "Negative cycle: ";
        for (int v : cycle)
            cout << v << ' ';
        cout << endl;
    }
}
```

7.21 Max Flow Dinic

```
// O(V^2 * E)
struct FlowEdge
{
    int v, u;
    long long cap, flow = 0;

    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic
{
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t)
    {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap = 1)
    {
        edges.emplace_back(v, u, cap);
        adj[v].push_back(m++);
        edges.emplace_back(u, v, 0);
        adj[u].push_back(m++);
    }

    bool bfs()
    {
        while (!q.empty())
        {
            int v = q.front();
            q.pop();
            for (int id : adj[v])
            {
                if (edges[id].cap == edges[id].flow)
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }

    long long dfs(int v, long long pushed)
    {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int &cid = ptr[v]; cid < (int)adj[v].size(); cid++)
        {
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u])
                continue;
            long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }

    long long flow()
    {
        long long f = 0;
        while (true)
        {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf))
            {
                f += pushed;
            }
        }
        return f;
    }

```

```
    bool dfs_path(int v, vector<int> &cur, vector<bool> &vis)
    {
        cur.push_back(v);
        vis[v] = true;

        if (v == t)
            return true;

        for (int id : adj[v])
        {
            if ((id & 1) || edges[id].flow <= 0)
                continue;

            int u = edges[id].u;

            if (!vis[u] && dfs_path(u, cur, vis))
            {
                edges[id].flow--;
                return true;
            }
        }

        cur.pop_back();
        return false;
    }

    vector<vector<int>> paths()
    {
        vector<vector<int>> ret;
        while (true)
        {
            vector<int> cur;
            vector<bool> vis(n, false);
            if (!dfs_path(s, cur, vis))
            {
                break;
            }
            ret.push_back(cur);
        }
        return ret;
    }
};
```

7.22 MCMF (Dijkstra)

```
// O(F * E log V)
struct MCMF
{
    struct Edge
    {
        int to;
        int cap, flow;
        int cost;
        int rev;
    };

    int n;
    vector<vector<Edge>> adj;
    vector<int> dist;
    vector<int> parent_edge, parent_node;
    vector<int> pot; // Potentials array for Dijkstra

    const int INF = numeric_limits<int>::max() / 2;

    MCMF(int n) : n(n), adj(n), pot(n, 0)
    {
    }

    void add_edge(int from, int to, int cap, int cost)
    {
        adj[from].push_back({to, cap, 0, cost, (int)adj[to].size()});
        // Reverse edge capacity is 0, cost is negated
        adj[to].push_back({from, 0, 0, -cost, (int)adj[from].size() - 1});
    }

    // Returns a pair of {max_flow, min_cost}
    pair<int, int> get_mcmf(int s, int t)
    {
        int max_flow = 0;
        int min_cost = 0;
        pot.assign(n, 0); // Initialize potentials to 0 (assumes initial costs >= 0)

        while (true)
        {
            dist.assign(n, INF);
            parent_node.assign(n, -1);
            parent_edge.assign(n, -1);

            // Priority queue for Dijkstra: {distance, node}
            priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq
 ;

            dist[s] = 0;
            pq.push({0, s});

            while (!pq.empty())
            {
                auto [d, u] = pq.top();
                pq.pop();

                // Skip outdated distance pairs
                if (d > dist[u])
                    continue;

                for (int i = 0; i < adj[u].size(); i++)
                {
                    Edge &e = adj[u][i];
                    if (e.cap - e.flow > 0)
                    {
                        // Reduced cost calculation: cost(u,v) + pot[u] - pot[v]
                        int reduced_cost = e.cost + pot[u] - pot[e.to];

                        if (dist[e.to] > dist[u] + reduced_cost)
                        {
                            dist[e.to] = dist[u] + reduced_cost;
                            parent_node[e.to] = u;
                            parent_edge[e.to] = i;
                            pq.push({dist[e.to], e.to});
```

```
                    }
                }
            }
        }

        // If we can't reach the sink, max flow is achieved
        if (dist[t] == INF)
            break;

        // Update potentials for the next iteration
        for (int i = 0; i < n; i++)
        {
            if (dist[i] != INF)
            {
                pot[i] += dist[i];
            }
        }

        // Find the bottleneck capacity along the shortest path
        int push = INF;
        int curr = t;
        while (curr != s)
        {
            int p = parent_node[curr];
            int idx = parent_edge[curr];
            push = min(push, adj[p][idx].cap - adj[p][idx].flow);
            curr = p;
        }

        // Push flow and accumulate cost
        max_flow += push;
        // pot[t] represents the true original cost of the shortest path from s to t
        min_cost += push * pot[t];

        // Update residual graph
        curr = t;
        while (curr != s)
        {
            int p = parent_node[curr];
            int idx = parent_edge[curr];
            int rev_idx = adj[p][idx].rev;
            adj[p][idx].flow += push;
            adj[curr][rev_idx].flow -= push;
            curr = p;
        }
    }

    return {max_flow, min_cost};
}

// DFS for flow decomposition (path reconstruction)
int dfs_path(int v, int t, int pushed, vector<int> &path, vector<int> &edge_ptr)
{
    if (v == t)
        return pushed;

    for (int &cid = edge_ptr[v]; cid < adj[v].size(); ++cid)
    {
        auto &edge = adj[v][cid];

        // Only traverse forward edges that have positive flow
        if (edge.flow > 0)
        {
            path.push_back(edge.to);
            int tr_pushed = dfs_path(edge.to, t, min(pushed, edge.flow), path, edge_ptr);

            if (tr_pushed > 0)
            {
                edge.flow -= tr_pushed; // Consume the flow to prevent reusing it
                return tr_pushed;
            }
            path.pop_back(); // Backtrack
        }
    }
    return 0;
}

// Reconstructs all paths taking flow from s to t
// Returns a vector of pairs: {bottleneck_flow, path_of_vertices}
// NOTE: Calling this function consumes the flow values in the graph.
vector<pair<int, vector<int>>> get_paths(int s, int t)
{
    vector<pair<int, vector<int>>> paths;
    vector<int> edge_ptr(n, 0); // Optimization to skip depleted edges

    while (true)
    {
        vector<int> path;
        path.push_back(s);
        int pushed = dfs_path(s, t, INF, path, edge_ptr);

        if (pushed == 0)
            break;
        paths.push_back({pushed, path});
    }
    return paths;
}
};
```

7.23 MCMF (SPFA)

```
struct Edge
{
    int to;
    int cost;
    int cap, flow, backEdge;
};

struct MCMF
{
    const int inf = 1000000010;
    int n;
    int s, t;
    vector<vector<Edge>> g;
    vector<int> state, pr, e;
    vector<int> d;
    deque<int> q;

```

```
    MCMF(int n, int s, int t) : n(n), s(s), t(t)
    {
        g.resize(n);
        state.resize(n);
        pr.resize(n);
        e.resize(n);
        d.resize(n);
    }

    void addEdge(int u, int v, int cap, int cost)
    {
        Edge e1 = {v, cost, cap, 0, (int)g[v].size()};
        Edge e2 = {u, -cost, 0, 0, (int)g[u].size()};
        g[u].push_back(e1);
        g[v].push_back(e2);
    }

    void bfs()
    {
        for (int i = 0; i < n; i++)
            state[i] = 2, d[i] = inf, pr[i] = -1;
        state[s] = 1;
        q.clear();
        q.push_back(s);
        d[s] = 0;
        while (!q.empty())
        {
            int v = q.front();
            q.pop_front();
            state[v] = 0;
            for (int i = 0; i < (int)g[v].size(); i++)
            {
                Edge c = g[v][i];
                if (c.flow >= c.cap || (d[c.to] <= d[v] + c.cost))
                    continue;
                int to = c.to;
                d[to] = d[v] + c.cost;
                pr[to] = v;
                e[to] = i;
                if (state[to] == 1)
                    continue;
                if (!state[to] || (!q.empty() && d[q.front()] > d[to]))
                    q.push_front(to);
                else
                    q.push_back(to);
                state[to] = 1;
            }
        }
    }

    pair<int, int> minCostMaxFlow()
    {
        int flow = 0;
        int cost = 0;
        while (true)
        {
            bfs();
            if (d[t] == inf)
                break;
            int it = t, addflow = inf;
            while (it != s)
            {
                addflow = min(addflow,
                              g[pr[it]][e[it]].cap - g[pr[it]][e[it]].flow);
                it = pr[it];
            }

            it = t;
            while (it != s)
            {
                g[pr[it]][e[it]].flow += addflow;
                g[it][g[pr[it]][e[it]].backEdge].flow -= addflow;
                cost += g[pr[it]][e[it]].cost * addflow;
                it = pr[it];
            }
            flow += addflow;
        }
        return {cost, flow};
    }
};
```

7.24 Bipartite Matching

```
struct BiMatching {
    vector<int> colAssign;
    vector<int> vis;
    vector<vector<int>> adjMat;
    int n, m;

    BiMatching(int n, int m) : n(n), m(m) {
        adjMat = vector<vector<int>>(n, vector<int>(m));
        colAssign = vector<int>(m, -1);
    }

    bool canMatch(int i) {
        for (int j = 0; j < m; j++) {
            if (adjMat[i][j] && !vis[j]) {
                vis[j] = 1;
                if (colAssign[j] == -1 || canMatch(colAssign[j])) {
                    colAssign[j] = i;
                    return true;
                }
            }
        }
        return false;
    }

    void addEdge(int u, int v) {
        adjMat[u][v] = 1;
    }

    int maxMatching() {
        int maxFlow = 0;
        for (int i = 0; i < n; i++) {
            vis = vector<int>(m, 0);
            if (canMatch(i))
                maxFlow++;
        }
```

```
        return maxFlow;
    }
};
```

7.25 Hopcroft-Karp

```
struct HK
{
    vector<vector<int>> adjList;
    vector<int> BtoA;
    HK(int n, int m)
    {
        adjList = vector<vector<int>>(n);
        BtoA = vector<int>(m, -1);
    }
    bool dfs(int a, int L, vector<int> &A, vector<int> &B)
    {
        if (A[a] != L)
            return 0;
        A[a] = -1;
        for (auto b : adjList[a])
        {
            if (B[b] == L + 1)
            {
                B[b] = 0;
                if (BtoA[b] == -1 or dfs(BtoA[b], L + 1, A, B))
                {
                    BtoA[b] = a;
                    return 1;
                }
            }
        }
        return 0;
    }
    inline void addEdge(int u, int v)
    {
        adjList[u].push_back(v);
    }
    inline int maxMatching()
    {
        int res = 0;
        vector<int> A(adjList.size()), B(BtoA.size()), cur, next;
        while (true)
        {
            fill(all(A), 0);
            fill(all(B), 0);
            cur.clear();
            for (auto a : BtoA)
                if (~a)
                    A[a] = -1;
            for (int a = 0; a < adjList.size(); a++)
                if (!A[a])
                    cur.push_back(a);
            for (int lay = 1;; lay++)
            {
                bool islast = 0;
                next.clear();
                for (auto a : cur)
                    for (auto b : adjList[a])
                    {
                        if (BtoA[b] == -1)
                            B[b] = lay, islast = 1;
                        else if (BtoA[b] != a and !B[b])
                            B[b] = lay, next.push_back(BtoA[b]);
                    }
                if (islast)
                    break;
                if (next.empty())
                    return res;
                for (auto a : next)
                    A[a] = lay;
                cur.swap(next);
            }
            for (int a = 0; a < adjList.size(); a++)
                res += dfs(a, 0, A, B);
        }
    }
};
```

7.26 Hungarian Algorithm

```
int a[N][N];
//O(n^3)

//Hungarian Algorithm for min cost barpartide matching, works also with negative costs
//All the nodes on the left should have an edge with all the nodes on the right (complete
     graph)
//a[1][1] represents the cost of the edge between node 1 in the left and node 1 in the right
//One based and every row should be matched with exactly one column


int hung() {
    vector<int> u(n + 1), v(m + 1), p(m + 1), way(m + 1);
    for (int i = 1; i <= n; ++i) {
        p[0] = i;
        int j0 = 0;
        vector<int> minv(m + 1, INF);
        vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], delta = INF, j1;
            for (int j = 1; j <= m; ++j)
                if (!used[j]) {
                    int cur = a[i0][j] - u[i0] - v[j];
                    if (cur < minv[j])
                        minv[j] = cur, way[j] = j0;
                    if (minv[j] < delta)
                        delta = minv[j], j1 = j;
                }
            for (int j = 0; j <= m; ++j)
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
                else
                    minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);
        do {
            int j1 = way[j0];
```

```
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    int cost = -v[0];
    return cost;
}
```

8 Strings

8.1 KMP Prefix Function

```
vector<int> prefix_function(string s)
{
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++)
    {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

bool match(string s , string t) {
    string cur = t + '#' + s;

    vector<int> pi = prefix_function(cur);

    for (int &i : pi)
        if ( i == t.size())
            return true;
    return false;
}
```

8.2 KMP Automaton

```

const int N = 1e5 + 3;
int nxt[N][26];
void build_automaton(string& v) {

    int m = v.size();
    vector<int> pi(m);

    for (int i = 1; i < m; i++) {
        int j = pi[i-1];
        while (j > 0 && v[i] != v[j]) j = pi[j-1];
        if (v[i] == v[j]) j++;
        pi[i] = j;
    }

    for (int i = 0; i <= m; i++) {
        for (int c = 0; c < 26; c++) {
            if (v[i] == (char)('a' + c))
                nxt[i][c] = i + 1;
            else if (i > 0)
                 nxt[i][c] = nxt[pi[i-1]][c];
            else
                nxt[i][c] = 0;
        }
    }
}
vector<pair<int,string>>adj[N];
ll ans = 0;
int az = 0;
void dfs(int node, int stat){
    for(auto &i:adj[node]){
        int nextState = stat;
        for(char &c:i.second)
        {
            nextState = nxt[nextState][c-'a'];
            ans += (nextState == az);
        }
        dfs(i.first,nextState);
    }
}

void solve()
{
    int n;cin>>n;
    for(int i=2,j;i<=n;i++){
        string s;
        cin>>j>>s;
        adj[j].push_back({i,s});
    }
    string a;
    cin>>a;
    build_automaton(a);
    az = a.size();
    dfs(1,0);
    cout<<ans<<'\n';
}
```

8.3 KMP Automaton 2

```

const int N = 55;
vector<vector<int>> build_automaton(string &v)
{
    int m = v.size();
    vector<int> pi(m + 1);
    vector<vector<int>> nxt(26, vector<int>(N, 0));
    for (int i = 1; i < m; i++)
    {
        int j = pi[i - 1];
        while (j > 0 && v[i] != v[j])
            j = pi[j - 1];
        if (v[i] == v[j])
            j++;
        pi[i] = j;
    }
```

```
    for (int i = 0; i <= m; i++)
    {
        for (int c = 0; c < 26; c++)
        {
            if (v[i] == (char)('a' + c))
                nxt[c][i] = i + 1;
            else if (i > 0)
                nxt[c][i] = nxt[c][pi[i - 1]];
            else
                nxt[c][i] = 0;
        }
    }
    return nxt;
}

void solve()
{
    string c, s, t;
    cin >> c >> s >> t;
    int n = c.size();
    // mx s mn t
    vector<vector<int>> nxtS = build_automaton(s);
    vector<vector<int>> nxtT = build_automaton(t);
    int ssz = s.size();
    int tsz = t.size();
    vector<vector<vector<int>>> dp(1004, vector<vector<int>>(53, vector<int>(53, -1e9)));
    dp[0][0][0] = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j <= ssz; j++)
        {
            for (int k = 0; k <= tsz; k++)
            {
                if (dp[i][j][k] != -1e9)
                {
                    for (int ch = 0; ch < 26; ch++)
                    {
                        if (c[i] == '*' || (char)('a' + ch) == c[i])
                        {
                            int nxt1 = nxtS[ch][j];
                            int nxt2 = nxtT[ch][k];
                            dp[i + 1][nxt1][nxt2] = max(dp[i + 1][nxt1][nxt2], dp[i][j][k] + (
 nxt1 == ssz) - (nxt2 == tsz));
                        }
                    }
                }
            }
        }
    }
    int mx = -1e9;
    for (int j = 0; j < 51; j++)
    {
        for (int k = 0; k < 51; k++)
        {
            mx = max(mx, dp[n][j][k]);
        }
    }

    cout << mx << '\n';
}
```

8.4 Z-Function

```
vector<int> z_function(string s)
{
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 1;
    for (int i = 1; i < n; i++)
    {
        if (i < r)
            z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n and s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

8.5 Manacher

```
vector<int> manacher(string &orginal)
{
    int n = size(orginal);
    string s = string(2 * n + 1, '#');
    for (int i = 0; i < n; i++)
        s[2 * i + 1] = orginal[i];
    n = 2 * n + 1;
    s.insert(s.begin(), '$');
    s.push_back('^');
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for (int i = 1; i <= n; i++)
    {
        p[i] = min(r - i, p[l + r - i]);
        while (s[i - p[i]] == s[i + p[i]])
            p[i]++;
        if (i + p[i] > r)
            r = i + p[i], l = i - p[i];
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}
```

8.6 Hashing

```

const int mod = 1e9 + 7;
const int N = 1e6;
pair<int, int> pw[N], inv[N];

int add(int a, int b)
{
    return (1ll * a + b + mod) % mod;
}
int mul(int a, int b)
{
```

```
    return (1ll * a * b) % mod;
}
int fastpow(int a, int b)
{
    if (!b)
        return 1;
    int hp = fastpow(a, b >> 1);
    hp = mul(hp, hp);
    return (b & 1 ? mul(hp, a) : hp);
}
int modInverse(int a)
{
    return fastpow(a, mod - 2);
}
void preHash()
{
    vector<int> primes;
    for (int i = 1; i <= 293; i++)
    {
        bool p = true;
        for (int j = 2; j * j <= i; j++)
        {
            if (i % j == 0)
                p = false;
        }
        if (p)
            primes.push_back(i);
    }
    srand(time(0));
    int b1 = primes[rand() % 62], b2 = primes[rand() % 62];
    pw[0] = inv[0] = {1, 1};
    int invB1 = modInverse(b1), invB2 = modInverse(b2);
    for (int i = 1; i < N; i++)
    {
        pw[i] = {mul(b1, pw[i - 1].first), mul(b2, pw[i - 1].second)};
        inv[i] = {mul(invB1, inv[i - 1].first), mul(invB2, inv[i - 1].second)};
    }
}
struct Hash
{
    vector<pair<int, int>> pre;
    pair<int, int> h(int num, int p)
    {
        return {mul(num, pw[p].first), mul(num, pw[p].second)};
    }
    int fa7l(char a)
    {
        return (a - 'a' + 1);
    }
    Hash(string &s)
    {
        pre.resize(s.size());
        for (int i = 0; i < s.size(); i++)
        {
            pre[i] = h(fa7l(s[i]), i);
            if (i)
                pre[i] = {add(pre[i].first, pre[i - 1].first), add(pre[i].second, pre[i - 1].
 second)};
        }
    }
    pair<int, int> getPrefix(int l, int r)
    {
        if (!l)
            return pre[r];
        return {mul(add(pre[r].first, -pre[l - 1].first), inv[l].first), mul(add(pre[r].second
  , -pre[l - 1].second), inv[l].second)};
    }
};
```

8.7 Shorter Hash

```
void preHash()
{
    auto now = chrono::high_resolution_clock::now();
    auto duration = chrono::duration_cast<chrono::microseconds>(now.time_since_epoch());
    srand(duration.count());
    int b[] = {31, 37};
    int pw[2][N];
    int inv[2][N];
    pw[0][0] = pw[1][0] = inv[0][0] = inv[1][0] = 1;
    int invB[] = {modInverse(b[0]), modInverse(b[1])};
    for (int j = 0; j < 2; j++)
        for (int i = 1; i < N; i++)
            pw[j][i] = mul(pw[j][i - 1], b[j]), inv[j][i] = mul(inv[j][i - 1], invB[j]);
}

void rabin()
{
    for (int j = 0; j < 2; j++)
        for (int i = 0; i < n; i++)
            pref[i + 1][j] = addm(pref[i][j], mul(doit(s[i]), pw[j][i]));
    function<Hash(int, int)> getHash = [&](int l, int r) -> Hash {
        Hash h = pref[r];
        h[0] = mul(addm(h[0], -pref[l - 1][0]), inv[0][l - 1]);
        h[1] = mul(addm(h[1], -pref[l - 1][1]), inv[1][l - 1]);
        return h;
    };
}
```

8.8 Multiset Hash

```
int b[2];
int pw[2][N];
int inv[2][N];
void precalc()
{
    b[0] = rand() + 2;
    b[1] = rand() + 2;
    pw[0][0] = pw[1][0] = inv[0][0] = inv[1][0] = 1;
    int invB[] = {modInverse(b[0]), modInverse(b[1])};
    for (int j = 0; j < 2; j++)
        for (int i = 1; i < N; i++)
            pw[j][i] = mul(pw[j][i - 1], b[j]), inv[j][i] = mul(inv[j][i - 1], invB[j]);
}
int doit(char c)
{
    return (c - 'a' + 1);
}
```

```cpp
typedef array<int, 2> Hash;
void solve()
{
    int n;
    vector<Hash> prefa(n + 1), prefb(n + 1);
    // A Hashing
    for (int i = 0; i < n; i++)
        for (int j = 0; j < 2; j++)
            prefa[i + 1][j] = addm(prefa[i][j], pw[j][a[i]]);
    // B Hashing
    for (int i = 0; i < n; i++)
        for (int j = 0; j < 2; j++)
            prefb[i + 1][j] = addm(prefb[i][j], pw[j][b[i]]);
}
```

8.9 Trie

```cpp
struct Trie {
    struct Node {
        int children[26]{};
        int f = 0;
    };
    vector<Node> trie;
    Trie() { trie.emplace_back(); }
    void insert(string &s) {
        int node = 0;
        for (auto &i : s) {
            int ch = (i - 'a');
            if (!trie[trie[node].children[ch]].f) {
                trie[node].children[ch] = trie.size();
                trie.emplace_back();
            }
            node = trie[node].children[ch];
            trie[node].f++;
        }
    }
    void erase(string &s) {
        int node = 0;
        for (auto &i : s) {
            int ch = (i - 'a');
            node = trie[node].children[ch];
            trie[node].f--;
        }
    }
    int query(string &s) {
        int node = 0;
        for (auto &i : s) {
            int ch = (i - 'a');
            if (!trie[trie[node].children[ch]].f) return 0;
            node = trie[node].children[ch];
        }
        return trie[node].f;
    }
};
```

8.10 Binary Trie

```cpp
struct BinaryTrie
{
    static constexpr int BITS = 30;
    struct Node
    {
        int go[2]{};
        int f[2]{};
    };
    vector<Node> trie;
    BinaryTrie()
    {
        trie.emplace_back();
    }
    void insert(int x)
    {
        int cur = 0;
        for (int i = BITS - 1; i >= 0; i--)
        {
            bool ch = x >> i & 1;
            if (!trie[cur].go[ch])
                trie[cur].go[ch] = trie.size(), trie.emplace_back();
            trie[cur].f[ch]++;
            cur = trie[cur].go[ch];
        }
    }
    void erase(int x)
    {
        int cur = 0;
        for (int i = BITS - 1; i >= 0; i--)
        {
            bool ch = x >> i & 1;
            trie[cur].f[ch]--;
            cur = trie[cur].go[ch];
        }
    }

    int get(int x)
    {
        // TODO : Logic Here
        int cur = 0;
        int ret = 0;
        for (int i = BITS - 1; i >= 0; i--)
        {
            bool ch = x >> i & 1;
            (trie[cur].f[ch ^ 1]) ? ret |= (1 << i) : ret;
            (trie[cur].f[ch ^ 1]) ? cur = trie[cur].go[ch ^ 1] : cur = trie[cur].go[ch];
        }
        return ret;
    }
};
```

8.11 Aho-Corasick

```cpp
#include <bits/stdc++.h>
using namespace std;

struct AhoCorasick
{
    static const int K = 26;

    struct Node
```

```cpp
{
    int nxt[K];
    int fail;

    // Pattern IDs ending at this node
    vector<int> ids;

    // Number of times this state is visited
    long long cnt = 0;

    // Used in forbidden-pattern problems
    bool bad = false;

    Node()
    {
        memset(nxt, -1, sizeof(nxt));
        fail = 0;
    }
};

vector<Node> trie;

AhoCorasick()
{
    trie.emplace_back(); // root
}

// Add pattern with ID
void addString(const string &s, int id)
{
    int v = 0;

    for (char ch : s)
    {
        int c = ch - 'a';

        if (trie[v].nxt[c] == -1)
        {
            trie[v].nxt[c] = trie.size();
            trie.emplace_back();
        }

        v = trie[v].nxt[c];
    }

    trie[v].ids.push_back(id);
    trie[v].bad = true; // if this is a forbidden pattern
}

void build()
{
    queue<int> q;

    // Root
    for (int c = 0; c < K; c++)
    {
        int u = trie[0].nxt[c];

        if (u == -1)
        {
            trie[0].nxt[c] = 0;
        }
        else
        {
            trie[u].fail = 0;
            q.push(u);
        }
    }

    while (!q.empty())
    {
        int v = q.front();
        q.pop();

        // If failure state is bad,
        // current state is also bad.
        if (trie[trie[v].fail].bad)
        {
            trie[v].bad = true;
        }

        for (int c = 0; c < K; c++)
        {

            int u = trie[v].nxt[c];

            if (u == -1)
            {
                // Follow failure transition
                trie[v].nxt[c] =
                    trie[trie[v].fail].nxt[c];
            }
            else
            {
                // Calculate failure link
                trie[u].fail =
                    trie[trie[v].fail].nxt[c];

                q.push(u);
            }
        }
    }
}

vector<long long> countOccurrences(const string &text, int n)
{

    // Visit states
    int v = 0;

    for (char ch : text)
    {
        int c = ch - 'a';

        v = trie[v].nxt[c];

        trie[v].cnt++;
    }
```

```
    /*
        Propagate counts through failure links.

        If:
            fail[v] = u

        then every occurrence represented by v
        is also an occurrence represented by u.
    */

    vector<int> order;
    order.reserve(trie.size());

    queue<int> q;
    q.push(0);

    while (!q.empty())
    {
        int v = q.front();
        q.pop();

        order.push_back(v);

        for (int c = 0; c < K; c++)
        {
            int u = trie[v].nxt[c];

            if (trie[u].fail == v && u != 0)
            {
                q.push(u);
            }
        }
    }

    // Reverse BFS order = deeper nodes first
    reverse(order.begin(), order.end());

    for (int v : order)
    {
        if (v == 0)
            continue;

        trie[trie[v].fail].cnt += trie[v].cnt;
    }

    // Answer for each pattern
    vector<long long> ans(n);

    for (int v = 0; v < (int)trie.size(); v++)
    {
        for (int id : trie[v].ids)
        {
            ans[id] = trie[v].cnt;
        }
    }

    return ans;
}
```

```
ll solve(int L)
{

    int states = trie.size();

    /*
        dp[state]

        Number of valid strings of current
        length ending in 'state'.
    */

    vector<ll> dp(states);
    vector<ll> ndp(states);

    dp[0] = 1;

    for (int len = 0; len < L; len++)
    {

        fill(ndp.begin(), ndp.end(), 0);

        for (int v = 0; v < states; v++)
        {

            if (dp[v] == 0)
                continue;

            for (int c = 0; c < K; c++)
            {

                int u = trie[v].nxt[c];

                // This creates a forbidden pattern
                if (trie[u].bad)
                    continue;

                ndp[u] += dp[v];

                if (ndp[u] >= MOD)
                    ndp[u] -= MOD;
            }
        }

        dp.swap(ndp);
    }

    ll ans = 0;

    for (int v = 0; v < states; v++)
    {
        ans += dp[v];

        if (ans >= MOD)
            ans -= MOD;
    }

    return ans;
```

```
    }
};
```

8.12 Aho-Corasick Start/End

```cpp
#include <bits/stdc++.h>

using namespace std;
using ll = long long;

#define int ll

struct AhoCorasick {
    static const int K = 26;

    struct Node {
        int nxt[K];
        int fail;

        // Pattern IDs ending at this node
        vector<int> ids;

        // Number of times this state is visited
        ll cnt = 0;

        ll match_cnt = 0;

        // Used in forbidden-pattern problems
        bool bad = false;

        Node() {
            memset(nxt, -1, sizeof(nxt));
            fail = 0;
        }
    };

    vector<Node> trie;

    AhoCorasick() {
        trie.emplace_back(); // root
    }

    // Add pattern with ID
    void addString(const string& s, int id) {
        int v = 0;

        for (char ch : s) {
            int c = ch - 'a';

            if (trie[v].nxt[c] == -1) {
                trie[v].nxt[c] = trie.size();
                trie.emplace_back();
            }

            v = trie[v].nxt[c];

        }

        trie[v].ids.push_back(id);
        trie[v].match_cnt++;
        trie[v].bad = true;// if this is a forbidden pattern
    }

    void build() {
        queue<int> q;

        // Root
        for (int c = 0; c < K; c++) {
            int u = trie[0].nxt[c];

            if (u == -1) {
                trie[0].nxt[c] = 0;
            } else {
                trie[u].fail = 0;
                q.push(u);
            }
        }

        while (!q.empty()) {
            int v = q.front();
            q.pop();

            // If failure state is bad,
            // current state is also bad.
            trie[v].match_cnt += trie[trie[v].fail].match_cnt;

            if (trie[trie[v].fail].bad) {
                trie[v].bad = true;
            }

            for (int c = 0; c < K; c++) {

                int u = trie[v].nxt[c];

                if (u == -1) {
                    // Follow failure transition
                    trie[v].nxt[c] =
                        trie[trie[v].fail].nxt[c];
                }
                else {
                    // Calculate failure link
                    trie[u].fail =
                        trie[trie[v].fail].nxt[c];

                    q.push(u);
                }
            }
        }
    }

};

void solve()
{
    string t;
    cin>>t;

    int n;
```

```
    cin>>n;

    AhoCorasick ac_forward;
    AhoCorasick ac_backward;

    for (int i = 0; i < n; i++) {
        string s;
        cin >> s;

        ac_forward.addString(s,i);

        reverse(s.begin(), s.end());
        ac_backward.addString(s,i);
    }

    ac_forward.build();
    ac_backward.build();

    int len = t.length();
    vector<int> ends_at(len, 0);
    vector<int> starts_at(len, 0);

    int v = 0;
    for (int i = 0; i < len; i++) {
        v = ac_forward.trie[v].nxt[t[i] - 'a'];
        ends_at[i] = ac_forward.trie[v].match_cnt;
    }

    v = 0;
    for (int i = len - 1; i >= 0; i--) {
        v = ac_backward.trie[v].nxt[t[i] - 'a'];
        starts_at[i] = ac_backward.trie[v].match_cnt;
    }

    ll tot = 0;
    for (int i = 0; i < len - 1; i++) {
        tot += 1LL * ends_at[i] * starts_at[i + 1];
    }

    cout << tot << "\n";

}

signed main()
{
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);

    int t = 1;
   // cin >> t;
    while (t--)
    {
        solve();
    }
}
```

8.13 Aho Matrix Trick

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const ll MOD = 1e9 + 7;

struct Matrix {

    int n;

    vector<vector<ll>> a;

    Matrix(int n, bool identity = false)
        : n(n),
          a(n, vector<ll>(n, 0)) {

        if (identity) {
            for (int i = 0; i < n; i++) {
                a[i][i] = 1;
            }
        }
    }

    Matrix operator*(const Matrix& other) const {

        Matrix res(n);

        for (int i = 0; i < n; i++) {

            for (int k = 0; k < n; k++) {

                if (a[i][k] == 0)
                    continue;

                for (int j = 0; j < n; j++) {

                    if (other.a[k][j] == 0)
                        continue;

                    res.a[i][j] +=
                        a[i][k] * other.a[k][j];

                    res.a[i][j] %= MOD;
                }
            }
        }

        return res;
    }
};


// ================================================
// Matrix exponentiation
// ================================================

Matrix power(Matrix base, long long exp) {

```

```cpp
    Matrix res(base.n, true);

    while (exp > 0) {

        if (exp & 1) {
            res = res * base;
        }

        base = base * base;

        exp >>= 1;
    }

    return res;
}


// ======================================================
// Aho-Corasick
// ======================================================

struct AhoCorasick {

    static const int K = 26;

    struct Node {

        int nxt[K];

        int fail;

        // Is this state forbidden?
        bool bad;

        Node() {

            memset(nxt, -1, sizeof(nxt));

            fail = 0;
            bad = false;
        }
    };

    vector<Node> trie;

    AhoCorasick() {

        trie.emplace_back();
    }

    // ------------------------------------------------
    // Add forbidden pattern
    // ------------------------------------------------

    void addString(const string& s) {

        int v = 0;

        for (char ch : s) {

            int c = ch - 'a';

            if (trie[v].nxt[c] == -1) {

                trie[v].nxt[c] =
                    trie.size();

                trie.emplace_back();
            }

            v = trie[v].nxt[c];
        }

        trie[v].bad = true;
    }

    // ------------------------------------------------
    // Build automaton
    // ------------------------------------------------

    void build() {

        queue<int> q;

        // Root
        for (int c = 0; c < K; c++) {

            int u = trie[0].nxt[c];

            if (u == -1) {

                trie[0].nxt[c] = 0;

            } else {

                trie[u].fail = 0;

                q.push(u);
            }
        }

        // BFS
        while (!q.empty()) {

            int v = q.front();
            q.pop();

            // If failure state is bad,
            // this state is also bad.
            if (trie[trie[v].fail].bad) {

                trie[v].bad = true;
            }

            for (int c = 0; c < K; c++) {

                int u = trie[v].nxt[c];

```

```
            if (u == -1) {

                trie[v].nxt[c] =
                    trie[trie[v].fail].nxt[c];

            } else {

                trie[u].fail =
                    trie[trie[v].fail].nxt[c];

                q.push(u);
            }
        }
    }
}

// ------------------------------------------------
// Build transition matrix
// ------------------------------------------------

Matrix getMatrix() {

    int states = trie.size();

    Matrix M(states);

    for (int v = 0; v < states; v++) {

        // We never use bad states.
        if (trie[v].bad)
            continue;

        for (int c = 0; c < K; c++) {

            int u = trie[v].nxt[c];

            // Don't transition into a bad state.
            if (trie[u].bad)
                continue;

            /*
                v --character--> u

                Therefore:
                    M[v][u]++

                ++ is important because multiple
                characters can lead to the same
                pair of states.
            */

            M.a[v][u]++;

            if (M.a[v][u] >= MOD)
                M.a[v][u] -= MOD;
        }
    }

    return M;
}

// ------------------------------------------------
// Solve
// ------------------------------------------------

ll solve(long long L) {

    int states = trie.size();

    Matrix M = getMatrix();

    // M^L
    Matrix P = power(M, L);

    /*
        Initially:

            dp = [1, 0, 0, ...]

        Therefore after L characters:

            dp[j] = P[0][j]
    */

    ll ans = 0;

    for (int state = 0; state < states; state++) {

        // Bad states are not reachable anyway,
        // but skipping them makes the meaning clear.
        if (trie[state].bad)
            continue;

        ans += P.a[0][state];

        if (ans >= MOD)
            ans -= MOD;
    }

    return ans;
}
};


int main() {

    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;

    long long L;

    cin >> n >> L;

    AhoCorasick ac;

    for (int i = 0; i < n; i++) {
```

```

        string s;

        cin >> s;

        ac.addString(s);
    }

    ac.build();

    cout << ac.solve(L) << '\n';

    return 0;
}
```

8.14 Hard Hash LCA

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <vector>
#define Kero                            \
    ios_base::sync_with_stdio(0);       \
    cout.tie(0);                        \
    cin.tie(0)
#define ll long long
#define ull unsigned long long
#define all(x) x.begin(), x.end()
#define allr(x) x.rbegin(), x.rend()
#define int ll
#define ld long double
#define pii pair<int, int>
#define endl '\n'
#define see(x) " [" << #x << " = " << (x) << "] "
using namespace std;
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update
    >;
template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
// order_of_key (k) : Number of items strictly smaller than k
// find_by_order(k) : K-th element in a set (counting from zero)
const int INF = 1e18, N = 2e5 + 5, mod = 1e9 + 7, LG = 20;
const ld pi = 2 * acos(0), eps = 1e-6;

void fileIO()
{
#ifndef ONLINE_JUDGE
    freopen("io/input.txt", "r", stdin);
    freopen("io/output.txt", "w", stdout);
#endif
}

// Coding is so easy if you simulate on paper first

map<int, int> facts;
int addm(int a)
{
    return ((a % mod) + mod) % mod;
}
template <typename... Args>
int addm(int a, Args... args)
{
    return (addm(a) + addm(args...)) % mod;
}
int subtm(int a, int b)
{
    return ((a % mod) - (b % mod) + mod) % mod;
}
int mul(int a)
{
    return ((a % mod) + mod) % mod;
}
template <typename... Args>
int mul(int a, Args... args)
{
    return (mul(a) * mul(args...)) % mod;
}
int fastpow(int b, int p)
{
    if (p == 0 || b == 1)
        return 1;
    int sub = fastpow(b, p / 2);
    if (p % 2 == 0)
        return mul(sub, sub);
    return mul(mul(sub, sub), b);
}
inline int modInverse(int a)
{
    return fastpow(a, mod - 2);
}
inline int divi(int a, int b)
{
    return mul(a, modInverse(b));
}
ll fact(ll a)
{
    if (a == 0ll)
        return 1;
    if (!facts.count(a))
        facts[a] = mul(fact(a - 1), a);
    return facts[a];
}
inline int nPr(int n, int r)
{
    if (n < r)
        return 0;
    return divi(fact(n), fact(n - r));
}
inline int nCr(int n, int r)
{
    if (n < r)
        return 0;
    return divi(nPr(n, r), fact(r));
```

```cpp
}
inline int starsNbars(int n, int r)
{
    return nCr(n + r - 1, r);
}
int b[] = {31, 37};
int pw[N][2];
int inv[N][2];
inline void precalc()
{
    pw[0][0] = pw[0][1] = inv[0][0] = inv[0][1] = 1;
    int invB[] = {modInverse(b[0]), modInverse(b[1])};
    for (int i = 1; i < N; i++)
        for (int j = 0; j < 2; j++)
            pw[i][j] = mul(pw[i - 1][j], b[j]), inv[i][j] = mul(inv[i - 1][j], invB[j]);
}
inline int doit(char c)
{
    return (c - 'a' + 1);
}
struct Hash
{
    int sum[2];
    int len;
    Hash()
    {
        sum[0] = sum[1] = 0;
        len = 0;
    }
    Hash(char c)
    {
        sum[0] = doit(c) * b[0];
        sum[1] = doit(c) * b[1];
        len = 1;
    }
    Hash operator+(const Hash &other) const
    {
        Hash ret = *this;
        ret.len = len + other.len;
        for (int i = 0; i < 2; i++)
        {
            ret.sum[i] = mul(ret.sum[i], pw[other.len][i]);
            ret.sum[i] = addm(ret.sum[i], other.sum[i]);
        }
        return ret;
    }
    // subtract The Sufix
    Hash operator-(const Hash &other) const
    {
        Hash ret = *this;
        ret.len = len - other.len;
        for (int i = 0; i < 2; i++)
        {
            ret.sum[i] = addm(ret.sum[i], -other.sum[i]);
            ret.sum[i] = mul(ret.sum[i], inv[other.len][i]);
        }
        return ret;
    }
    bool operator==(const Hash &other) const
    {
        return len == other.len and sum[0] == other.sum[0] and sum[1] == other.sum[1];
    }
    Hash sub_pref(const Hash &other) const
    {
        Hash ret = *this;
        ret.len = len - other.len;
        for (int i = 0; i < 2; i++)
        {
            ret.sum[i] = addm(ret.sum[i], -mul(other.sum[i], pw[ret.len][i]));
        }
        return ret;
    }
    static vector<Hash> getHash(string &str)
    {
        vector<Hash> ret;
        ret[0] = Hash(str[0]);
        for (int i = 1; i < str.size(); i++)
            ret[i] = ret[i - 1] + Hash(str[i]);
        return ret;
    }
    static Hash getRange(vector<Hash> &hsh, int l, int r)
    {
        l--;
        Hash ret = hsh[r];
        if (l)
            ret = ret - hsh[l];
        return ret;
    }
};


string str;
int deep[N];
Hash deepHash[N];
Hash deepHash2[N];
vector<int> g[N];
int up[N][LG];
void init(int n)
{
    for(int i = 0; i <= n; i++)
    {
        memset(up[i], 0, sizeof up[i]);
        deepHash[i] = Hash();
        deepHash2[i] = Hash();
        g[i].clear();
    }
}
void dfs(int cur)
{
    for (auto &ch : g[cur])
    {
        if (ch == up[cur][0])
            continue;
        deep[ch] = deep[cur] + 1;
        up[ch][0] = cur;
        deepHash[ch] = deepHash[cur] + Hash(str[ch]);
        deepHash2[ch] = Hash(str[ch]) + deepHash2[cur];
        for (int lg = 1; lg < LG and up[ch][lg - 1]; lg++)
```

```
            up[ch][lg] = up[up[ch][lg - 1]][lg - 1];
        dfs(ch);
    }
}
inline int kthAnc(int u, int k)
{
    for (int lg = LG - 1; lg >= 0; lg--)
        if ((k >> lg) & 1)
            u = up[u][lg];
    return u;
}
inline int getLca(int u, int v)
{
    if (deep[v] > deep[u])
        swap(v, u);
    u = kthAnc(u, deep[u] - deep[v]);
    if (u == v)
        return u;
    for (int lg = LG - 1; lg >= 0; lg--)
        if (up[u][lg] != up[v][lg])
            u = up[u][lg], v = up[v][lg];
    return up[u][0];
}
inline int getDist(int u, int v)
{
    return deep[u] + deep[v] - 2 * deep[getLca(u, v)];
}
inline Hash getPath(int u, int v)
{
    int lca = getLca(u, v);
    Hash hsh_v = deepHash[v].sub_pref(deepHash[up[lca][0]]);
    Hash hsh_u = deepHash2[u] - deepHash2[lca];
    // cout << see(lca) << endl;
    // cout << hsh_u.sum[0] << endl;
    return hsh_u + hsh_v;
}
inline int kthInPath(int u, int v, int k)
{
    int lca = getLca(u, v);
    int dist = deep[u] + deep[v] - 2 * deep[lca];
    if (deep[u] - deep[lca] < k)
        return kthAnc(v, dist - k);
    else
        return kthAnc(u, k);
}
inline int query(int a, int b, int c, int d)
{
    int d1 = getDist(a, b);
    int d2 = getDist(c, d);
    int dist = min(d1, d2);
    // cout << getPath(a, b).sum[0] << endl;
    // cout << getPath(c, d).sum[0] << endl;
    int l = 0, r = dist, mid, ans = -1;
    while(l <= r)
    {
        mid = l + (r - l) / 2;
        Hash hsh_a = getPath(a, kthInPath(a, b, mid));
        Hash hsh_c = getPath(c, kthInPath(c, d, mid));
        if(hsh_a == hsh_c)
        {
            l = mid + 1;
        }
        else
        {
            ans = mid;
            r = mid - 1;
        }
    }
    if(ans == -1)
    {
        if(d1 > d2)
            return 1;
        else if (d1 < d2)
            return 2;
        return 0;
    }
    int kth_a = kthInPath(a, b, ans);
    int kth_c = kthInPath(c, d, ans);
    if(str[kth_a] > str[kth_c])
        return 1;
    return 2;
}
inline void solve()
{
    int n;
    cin >> n;
    init(n);
    // Hash hsh;
    // str = "bab";
    // for(int i = 0; i < 3; i++)
    //     hsh = hsh + Hash(str[i]);
    // cout << hsh.sum[0] << endl;
    // hsh = hsh - Hash();
    // cout << hsh.sum[0] << endl;
    // cout << endl;
    cin >> str;
    str = " "+ str;
    for(int i = 2, u, v; i <= n; i++)
        cin >> u >> v, g[u].push_back(v), g[v].push_back(u);
    deep[1] = 0;
    deepHash[1] = Hash(str[1]);
    deepHash2[1] = Hash(str[1]);
    dfs(1);
    // cout << getPath(1, 3).sum[0] << endl;
    int q;
    cin >> q;
    while (q--) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        cout << query(a, b, c, d) << endl;
    }
}

signed main()
{
    Kero;
    fileIO();
    precalc();
```

```
    int t = 1;
    cin >> t;
    while (t--)
        solve();
    return 0;
}
```

9 Math & Geometry

9.1 Max Manhattan Distance

- For a set of points: $\max(|\max(u) - \min(u)|, |\max(v) - \min(v)|)$
- Where $u = x + y$ and $v = x - y$

9.2 Linear Sieve

```
const int N = 10000000;
vector<int> lp(N+1);
vector<int> pr;

 for (int i=2; i <= N; ++i) {
   if (lp[i] == 0) {
      lp[i] = i; pr.push_back(i);
    }
     for (int j = 0; i * pr[j] <= N; ++j)
       {
       lp[i * pr[j]] = pr[j];
        if (pr[j] == lp[i])
         { break; }
    }
}
```

9.3 Miller-Rabin Primality

```
using u64 = uint64_t;
using u128 = __uint128_t;
u64 binpower(u64 base, u64 e, u64 mod)
{
    u64 result = 1;
    base %= mod;
    while (e)
    {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}
bool check_composite(u64 n, u64 a, u64 d, int s)
{
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++)
    {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
}
bool MillerRabin(u64 n)
{
    // returns true if n is prime, else returns false.
    if (n < 2)
        return false;
    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0)
    {
        d >>= 1;
        r++;
    }
    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}
```

9.4 Phi & Mobius Sieve

```
int phi[N + 1];
int mu[N + 1];
bool vis[N + 1];
vector<int> divs[N + 1];

void pre()
{
    for (int i = 1; i <= N; i++)
        phi[i] = i, mu[i] = 1;
    for (int i = 2; i <= N; i++)
        if (phi[i] == i)
            for (int j = i; j <= N; j += i)
                phi[j] -= phi[j] / i;
    for (int i = 2; i <= N; i++)
        if (!vis[i])
            for (int j = i; j <= N; j += i)
                vis[j] = 1, mu[j] = (j % (i * i) == 0 ? 0 : (mu[j] * -1));
}
```

9.5 Phi / Sqrt

```
int phi(int x)
{
  int ret = x;
  for (int i = 2; i * i <= x; i++)
  {
    if (x % i == 0)
    {
      while (x % i == 0)
        x /= i;
```

```
      ret -= ret / i;
    }
  }
  if (x > 1)
    ret -= ret / x;
  return ret;
}
```

9.6 Extended Euclidean

```
int euclid(int a, int b, int &x, int &y)
{
    if (a < 0 || b < 0)
    {
        int g = euclid(abs(a), abs(b), x, y);
        if (a < 0)
            x *= -1;
        if (b < 0)
            y *= -1;
        return g;
    }
    if (b == 0)
    {
        x = 1, y = 0;
        return a;
    }
    int x1, y1;
    int g = euclid(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}
int find_sol(int a, int b, int c, int &x0, int &y0, bool &found)
{
    int g = euclid(abs(a), abs(b), x0, y0);
    found = 1;
    if (c % g)
    {
        found = 0;
        return g;
    }
    x0 *= c / g;
    y0 *= c / g;
    return g;
}
```

9.7 Linear Diophantine

```
void shift_solution(int &x, int &y, int a, int b, int cnt)
{
    x += cnt * b;
    y -= cnt * a;
}
int find_all_solutions(int a, int b, int c, int minx, int maxx, int miny, int maxy)
{
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;
    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;
    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;
    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;
    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;
    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    int rx2 = x;
    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);
    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}
```

9.8 Find All Diophantine Solutions

```
int gcd(int a, int b, int &x, int &y)
{
    if (b == 0)
    {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g)
{
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g)
    {
        return false;
    }
    x0 *= c / g;
    y0 *= c / g;
    if (a < 0)
        x0 = -x0;
```

```
    if (b < 0)
        y0 = -y0;
    return true;
}
void shift_solution(int &x, int &y, int a, int b, int cnt)
{
    x += cnt * b;
    y -= cnt * a;
}
int find_all_solutions(int a, int b, int c, int minx, int maxx, int miny, int maxy)
{
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;
    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;
    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;
    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;
    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;
    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    int rx2 = x;
    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);
    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}
```

9.9 GCD Trick

```
int n, m, k;
cin >> n >> m >> k;

// f[x] = frequency of value x in the array
vector<ll> f(m + 1);

for (int i = 0; i < n; i++)
{
    int x;
    cin >> x;
    f[x]++;
}

/*
    div[i] = number of elements divisible by i.

    We iterate over multiples of i:

        i, 2i, 3i, ...

    and add their frequencies.
*/
vector<ll> div(m + 1);

for (int i = 1; i <= m; i++)
{
    for (int j = i; j <= m; j += i)
    {
        div[i] += f[j];
    }
}

/*
    g[i] = number of NON-EMPTY subsequences
            whose gcd is exactly i.

    First:

        2^div[i] - 1

    counts all non-empty subsequences where every
    selected element is divisible by i.

    Their gcd can be:

        i, 2i, 3i, ...

    Therefore we subtract the answers for all
    proper multiples of i.
*/

vector<ll> g(m + 1);

for (int i = m; i >= 1; i--)
{

    // All non-empty subsequences consisting of
    // elements divisible by i.
    g[i] = (modPow(2, div[i]) - 1 + MOD) % MOD;

    /*
        Remove subsequences whose gcd is a proper
        multiple of i.
    */
    for (int j = 2 * i; j <= m; j += i)
    {
        g[i] -= g[j];

        if (g[i] < 0)
```

```
            g[i] += MOD;
    }
}
```

9.10 Count Number of Divisors

```
bool is_prime_square(int n)
{
    int num = sqrtl(n);
    return num * num == n && MillerRabin(num);
}
int count_divisors(int n)
{
    // can also be used to get:
    // sum and product of the divisors
    int ans = 1;
    for (auto &p : primes)
    {
        if (p * p * p > n)
            break;
        if (n % p != 0)
            continue;
        int cnt = 0;
        while (n % p == 0)
        {
            cnt++;
            n /= p;
        }
        ans *= (cnt + 1);
    }
    if (MillerRabin(n))
        ans *= 2;
    else if (is_prime_square(n))
        ans *= 3;
    else if (n != 1)
        ans *= 4;
    return ans;
}
```

9.11 Count Subarrays with LCM = q

```
//count all arrays with lcm(a[i], a[i+1]) == k

inline vector<int> getPrimeFactors(int x)
{
    map<int, int> freq;
    vector<int> ret;
    for (int i = 2; i * i <= x; i++)
        while (x % i == 0)
            freq[i]++, x /= i;
    if(x > 1)
        freq[x]++;
    for(auto &[p, frq] : freq)
        ret.push_back(frq);
    return ret;
}
void solve()
{
    int n, k;
    cin >> n >> k;
    vector<int> factors = getPrimeFactors(k);
    ll ans = 1;
    Matrix S, T;
    for(auto &frq : factors)
    {
        S = {{0, 1}};
        T = {{0, 1},
             {frq, 1}};
        T = power(T, n);
        S = mul(S, T);
        S[0][1]  = (1LL * S[0][0] + S[0][1]) % mod;
        ans *= S[0][1];
        ans %= mod;
    }
    cout << ans << endl;
}
```

9.12 Counting (Combinations)

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1e9 + 7;
const int MAXN = 1e6 + 5;   // adjust as needed

ll fact[MAXN], invfact[MAXN];


// Fast Power (a^b % MOD)
ll fastPow(ll a, ll b) {
    ll res = 1;
    a %= MOD;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

// Modular Inverse (MOD must be prime)
ll modinv(ll a) {
    return fastPow(a, MOD - 2);
}

// Precompute factorials and inverse factorials
void init_factorials() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++)
        fact[i] = fact[i - 1] * i % MOD;

    invfact[MAXN - 1] = modinv(fact[MAXN - 1]);
    for (int i = MAXN - 2; i >= 0; i--)
        invfact[i] = invfact[i + 1] * (i + 1) % MOD;
}

// nCr (Combination)
```

```cpp
ll nCr(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
}
bool nCr(int n, int r)
{
return ((n & r) == r);
}

// nPr (Permutation)
ll nPr(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[n - r] % MOD;
}

// Combination with Repetition
// (n multichoose r) = (n+r-1)Cr
ll nCr_rep(ll n, ll r) {
    return nCr(n + r - 1, r);
}

// Stars and Bars
// Distribute k identical items into n distinct boxes
ll stars_and_bars(ll n, ll k) {
    // (k + n - 1)C(n - 1)
    return nCr(k + n - 1, n - 1);
}

// Stars and Bars (at least 1 in each box)
ll stars_and_bars_at_least_one(ll n, ll k) {
    // Give 1 to each first
    if (k < n) return 0;
    return nCr(k - 1, n - 1);
}
```

9.13 Matrix Power

```cpp
const int mod = 1e9 + 7;
using Row = vector<long long>;
using Matrix = vector<Row>;

Matrix mul(Matrix &a, Matrix &b) {
    int n = a.size(), m = a[0].size(), k = b[0].size();
    Matrix res(n, Row(k));
    for(int i = 0; i < n; ++i)
        for(int j = 0; j < k; ++j)
            for(int o = 0; o < m; ++o) {
                res[i][j] += 1ll * a[i][o] * b[o][j] % mod;
                if(res[i][j] >= mod) res[i][j] -= mod;
                if(res[i][j] < 0) res[i][j] += mod;
            }
    return res;
}

Matrix power(Matrix a, long long b) {
    int n = a.size();
    Matrix res(n, Row(n));
    for(int i = 0; i < n; ++i) res[i][i] = 1;

    while(b) {
        if(b&1) res = mul(res, a);
        a = mul(a, a), b >>= 1;
    }

    return res;
}
```

9.14 Big Integer

```cpp
__int128 read()
{
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9')
    {
        if (ch == '-')
            f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9')
    {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

void print(__int128 x)
{
    if (x < 0)
    {
        putchar('-');
        x = -x;
    }
    if (x > 9)
        print(x / 10);
    putchar(x % 10 + '0');
}

bool cmp(__int128 x, __int128 y) {
    return x > y;
}
```

9.15 Solve Quadratics

```cpp
int quadRoots(double a, double b, double c, pair<double, double> &out)
{
    assert(a != 0);
    double disc = b * b - 4 * a * c;
    if (disc < 0)
        return 0;
    double sum = (b >= 0) ? -b - sqrt(disc) : -b + sqrt(disc);
    out = {sum / (2 * a), sum == 0 ? 0 : (2 * c) / sum};
    return 1 + (disc > 0);
}
```

9.16 Convex Hull

```cpp
struct point
{
    ll x, y;
    point(ll x=0, ll y=0) : x(x), y(y)
    {
    }
    point operator-(point other)
    {
        return point(x - other.x, y - other.y);
    }
    bool operator<(const point &other) const
    {
        return x != other.x ? x < other.x : y < other.y;
    }
};
ll cross(point a, point b)
{
    return a.x * b.y - a.y * b.x;
}
ll dot(point a, point b)
{
    return a.x * b.x + a.y * b.y;
}
struct sortCCW
{
    point center;

    sortCCW(point center) : center(center)
    {
    }
    bool operator()(point a, point b)
    {
        ll res = cross(a - center, b - center);
        if (res)
            return res > 0;
        return dot(a - center, a - center) < dot(b - center, b - center);
    }
};
vector<point> hull(vector<point> v)
{
    sort(v.begin(), v.end());
    sort(v.begin() + 1, v.end(), sortCCW(v[0]));
    v.push_back(v[0]);
    vector<point> ans;
    for (auto i : v)
    {
        int sz = ans.size();
        while (sz > 1 && cross(i - ans[sz - 1], ans[sz - 2] - ans[sz - 1]) <= 0)
            ans.pop_back(), sz--;
        ans.push_back(i);
    }
    ans.pop_back();
    return ans;
}
```

9.17 Geometry

```cpp
#include <bits/stdc++.h>
using namespace std;

#define T long double
#define point complex<T>
#define Y imag()
#define X real()
/*
abs(): Returns the magnitude (absolute value) of the complex number.
arg(): Returns the argument (polar angle) of the complex number.
conj(): Returns the complex conjugate of the complex number
hypot(c,d): Takes two lengths and returns the hypotenus
polar(c,theta): Return a complex with these polar cords
*/
const double EPS = 1e-9;
const double PI = 3.14159265358979323846;
point mkvec(point a,point b){
    return b - a;
}

T cross(point a, point b) {
    return a.X*b.Y - a.Y*b.X;
}

T clamp(T val){
    return min((T)1.0 , max( (T)-1.0 , val));
}

T dot(point a, point b) {
    return (conj(a) * b).real();
}
// dotProduct = |A| * |B| * cos(Theta)
T calcAngle(point v1,point v2){
    return acos(clamp(dot(v1,v2)/abs(v1)/abs(v2)));
}
// Convert degrees to radians
T toRad(T deg) {
    return deg * PI / 180.0;
}

// Rotation CCW (use negative angle for CW)
point rotate(point p, point pivot, T angle) {
    return (p - pivot) * polar((T)1.0, angle) + pivot;
}
/*

// Rotation CCW (use negative angle for CW)
point rotate(point p, point pivot,int ang) {
    p = (p - pivot);
    if(ang == 90)
        p = point(-p.Y,p.X);
    else if(ang == -90) {
        p = point(p.Y,-p.X);
    }
     return p + pivot;
}
*/
point readPoint(){
    int x,y;
```

```
    cin>>x>>y;
    return point(x,y);
}

T angleCCW(point v1, point v2) {
    T a1 = atan2(v1.Y, v1.X);
    T a2 = atan2(v2.Y, v2.X);
    T angle = a2 - a1;

    if (angle < -EPS) angle += 2.0 * PI;
    return angle;
}

// 1 - left - ccw
int orientation (point a,point b,point c){
    point ab = mkvec(a,b);
    point ac = mkvec(a,c);
    T cr = cross(ab,ac);
    if(cr > 0) return 1;
    if(cr < 0) return -1;
    return 0;

}

//////////////////////////Lines////////////////////

// normal equation => Ax + By + C = 0
// slop-intercept form => y = Mx + B
// Essential Core Functions
//double dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
//double cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
//double magnitude(Point a) { return hypot(a.x, a.y); }
//double dist(Point a, Point b) { return hypot(a.x - b.x, a.y - b.y); }
// summation of angles for regular n-Gon
// In degrees: (n - 2) × 180°

// ========================================
// 2. LINE STRUCT (Ax + By + C = 0)
// ========================================
struct Line {
    double a, b, c;
    double slope,yinter;
    Line(double a = 0, double b = 0, double c = 0) : a(a), b(b), c(c) {}

    // Build line from two points
    void fromTwoPoints(point p1, point p2) {
        a = p1.Y - p2.Y;
        b = p2.X - p1.X;
        c = -a * p1.X - b * p1.Y;
    }

    // Build line from a point and a normal vector
    void fromPointAndNormal(point p, point normal) {
        a = normal.X;
        b = normal.Y;
        c = -a * p.X - b * p.Y;

    }
    void slop_inter(){
        slope = a/b;
        yinter = c/b;
    }
};

// ========================================
// 3. MULTI-FUNCTION UTILITIES
// ========================================

// Perpendicular distance from Point to Line
T distToLine(point p, Line l) {
    return abs(l.a * p.X + l.b * p.Y + l.c) / hypot(l.a, l.b);
}

// Line-Line Intersection
// Returns false if lines are parallel/coincident, true if they intersect
bool intersect(Line l1, Line l2, point& out_intersection) {
    T zn = l1.a * l2.b - l2.a * l1.b;
    if (abs(zn) < EPS) return false;

    out_intersection = point (- (l1.c * l2.b - l2.c * l1.b) / zn, -(l1.a * l2.c - l2.a * l1.c)
      / zn);

    return true;
}

// Area of any simple polygon using Shoelace Formula
T polygonArea(const vector<point>& vertices) {
    T area = 0;
    int n = vertices.size();
    for (int i = 0; i < n; i++) {
        area += cross(vertices[i], vertices[(i + 1) % n]);
    }
    return abs(area) / 2.0;
}
// T t = dot(mkvec(a,p),mkvec(a,b)) / dot(mkvec(a,b),mkvec(a,b));
// t < 0 -> less than a
// t > 1 -> more than b
// else the image is on the line
bool onSegment(point a, point b, point c) {
    return (c.X >= min(a.X, b.X) && c.X <= max(a.X, b.X) &&
            c.Y >= min(a.Y, b.Y) && c.Y <= max(a.Y, b.Y));
}
//////// CIRCLES/////////////////
T circumcircleRadius(point p1, point p2, point p3) {
    T a = abs(p1 - p2);
    T b = abs(p2 - p3);
    T c = abs(p3 - p1);

    T area = polygonArea({p1, p2, p3});

    // Avoid division by zero if the points are collinear (on the same line)
    if (area < EPS) return -1.0;

    return (a * b * c) / (4.0 * area);
}

vector<point> intersectCircles(point c1, T r1, point c2, T r2) {
    T d = abs(c2 - c1);
```

```
177
178     // Case 1: Circles are too far apart, or one is strictly inside the other
179     if (d > r1 + r2 + EPS || d < abs(r1 - r2) - EPS || d < EPS) {
180         return {}; // No intersection
181     }
182
183     // Case 2: They intersect or are tangent
184     // Using Law of Cosines to find the angle from c1: r2^2 = r1^2 + d^2 - 2*r1*d*cos(a)
185     T cos_a = (r1 * r1 + d * d - r2 * r2) / (2.0 * r1 * d);
186     cos_a = clamp(cos_a); // Ensure value is between [-1, 1] before acos
187     T angle = acos(cos_a);
188
189     // Create a vector from c1 pointing exactly to c2, then scale its length to r1
190     point v = (c2 - c1) / d * r1;
191
192     point p1 = c1 + v * polar((T)1.0, angle);
193     point p2 = c1 + v * polar((T)1.0, -angle);
194
195     if (abs(p1 - p2) < EPS)
196         return {p1};
197
198     return {p1, p2};
199 }
200
201 //not tested
202 // Returns the center of the circle passing through 3 non-collinear points (Circumcenter)
203 point circumcenter(point a, point b, point c) {
204     point ab = b - a; // Vector from A to B
205     point ac = c - a; // Vector from A to C
206
207     // Calculate the determinant (which is 2 * cross product)
208     T D = 2.0 * cross(ab, ac);
209
210     // If D is 0, it means the 3 points are collinear (on the same line),
211     // so a circle cannot be drawn through them.
212     if (abs(D) < EPS) {
213         return point(1e18, 1e18); // Return a dummy/infinity point as an error flag
214     }
215
216     // In <complex>, std::norm(v) returns the SQUARED magnitude (x^2 + y^2)
217     // Don't confuse it with abs() which returns the actual distance.
218     T ab_sq = norm(ab);
219     T ac_sq = norm(ac);
220
221     // Apply Cramer's rule to find the center relative to 'a'
222     T cx = (ac.Y * ab_sq - ab.Y * ac_sq) / D;
223     T cy = (ab.X * ac_sq - ac.X * ab_sq) / D;
224
225     // Add 'a' back to shift the center to its true absolute position
226     return a + point(cx, cy);
227 }
228
229 int pointInPolygon(vector<point>polygon,point p){
230     int n = polygon.size();
231
232     for(int i=0;i<n;i++){
233             if( (orientation(polygon[i],polygon[(i+1)%n],p) == 0) && onSegment(polygon[i],
        polygon[(i+1)%n],p))
234             {
235                 //"BOUNDARY\n";
236                 return 1;
237             }
238         }
239
240         bool inside = false;
241         for(int i=0;i<n;i++){
242             point a = polygon[i],b = polygon[(i+1)%n];
243             if(a.Y > b.Y)
244                 swap(a,b);
245             if( p.Y >= a.Y && p.Y < b.Y )
246             {
247                 if(orientation(a,b,p) == 1){
248                     inside = !inside;
249                 }
250             }
251
252         }
253        // cout << (inside? "INSIDE\n":"OUTSIDE\n");
254         if(inside)
255             return 2;
256         return 0;
257 }
258
259 // line xy , ab
260 bool lineSegmentsIntersect(point a,point b,point x,point y){
261     long long r1 = orientation(a,b,x);
262     long long r2 = orientation(a,b,y);
263     long long r3 = orientation(x,y,a);
264     long long r4 = orientation(x,y,b);
265
266     if(r1*r2 < 0 && r3*r4 < 0)
267         return 1;
268     else if( (r1 == 0 && onSegment(a,b,x)) ||
269         (r2 == 0 && onSegment(a,b,y)) ||
270         (r3 == 0 && onSegment(x,y,a)) ||
271         (r4 == 0 && onSegment(x,y,b))
272         ){
273             return 1;
274     }
275     return 0;
276 }
277
278
279 // check if the all angles are 90 in quad
280 bool four90(vector<point> v){
281     for(int i=0;i<4;i++)
282         if(dot(v[(i+1)%4]-v[i],v[(i+2)%4]-v[(i+1)%4]) != 0)
283             return false;
284
285     return true;
286
287 }
288
289 int main()
290 {
291
292    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
293
```

```
    function<bool(point,point,point,point)> formSquare = [](point a,point b,point c,point d)
     -> bool{
        vector<T>dis;
        dis.push_back(norm(a-b));
        dis.push_back(norm(a-c));
        dis.push_back(norm(a-d));
        dis.push_back(norm(d-b));
        dis.push_back(norm(d-c));
        dis.push_back(norm(b-c));
        sort(dis.begin(),dis.end());
        return dis[0] == dis[1] && dis[1] == dis[2] && dis[2] == dis[3] && dis[4] == dis[5] &&
      dis[0] > EPS;
    };

    return 0;
}
```

10 Dynamic Programming

10.1 LIS

```
int n, mx = 0;
int arr[N] = {};
vector<int> dp(N, INF);
void solve()
{
    // memset(dp, -1, sizeof dp);
    cin >> n;
    for (size_t i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    dp[0] = -INF;
    for (int i = 0; i < n; i++)
    {
        int l = upper_bound(dp.begin(), dp.end(), arr[i]) - dp.begin();
        if (dp[l - 1] < arr[i] && arr[i] < dp[l])
            dp[l] = arr[i];
    }
    for (int l = 0; l <= n; l++)
    {
        if (dp[l] < INF)
            mx = l;
    }
    cout << mx << endl;
}
```

10.2 LIS with DS

```
void solve()
{
    int n, k;
    cin >> n >> k;
    vector<int> v(n);
    int mx = 0;
    for (auto &x : v)
        cin >> x, mx = max(mx, x);
    vector<BIT> dp(k + 1, BIT(mx + 1));

    for (int i = 0; i < n; i++)
    {
        dp[1].update(v[i] + 1, 1);
        for (int j = 1; j < k; j++)
        {
            int add = dp[j].pref(v[i]);
            dp[j + 1].update(v[i] + 1, add);
        }
    }
    cout << dp[k].pref(mx + 1);
}
```

10.3 Bitset DP

```
class Solution {
public:

    int maxTotalReward(vector<int>& a) {
        int n = a.size();
        a.push_back(-1);
        sort(a.begin(), a.end());
        const int MX = 1e5;
        bitset<MX> dp, allones;
        dp[0] = 1;
        int j = 0;
        for(int i = 1; i <= n; i++)
        {
            // for(int j = 0; j < MX; j++)
            // {
            //     dp[i][j] = dp[i - 1][j];
            //     if(2 * a[i] > j and j - a[i]>=0 )
            //     dp[i][j] = max(dp[i][j], dp[i - 1][j - a[i]] + a[i]);
            // }
            while(a[i] != j)
                allones[j++]=1;
            dp |= ((dp & allones) << a[i]);
        }
        for(int i = MX - 1; ~i; i--)
            if(dp[i])
                return i;
        return 0;
    }
};
```

10.4 Digit DP

```
const int mod = 998244353;
ll add(ll a, ll b) {
    return ( a + b + mod) % mod;
}
ll mul(ll a, ll b) {
    return (a * b) % mod;
}
struct Node{
    ll cnt,sum;
};
bool memo[2][2][(1<<10)][20];
```

```
Node dp [2][2][(1<<10)][20]; //ru,rd,lz,msk (taken digits) , i = n
string l,r;
int n,k;

ll pow10[20];
Node rec(bool ru,bool rd,int msk,int i){

    if(i == n){
        //
        return {1,0};
    }

    Node &ret = dp[ru][rd][msk][i];
    if(memo[ru][rd][msk][i])
        return ret;

    short up = ru ? r[i] - '0' : 9;
    short down = rd ? l[i] - '0' : 0;

    ret = { 0 , 0};

    for(ll d=down ; d<=up ; d++){
        ll nexMsk = msk;
        if( !(msk == 0 && d == 0)) // not leading zeros
            nexMsk = msk | (1<<d);

        if( __builtin_popcount(nexMsk) <= k ){
            Node temp = rec(ru && d==up , rd && d==down , nexMsk ,i+1);

            ret.cnt = add(ret.cnt , temp.cnt);
            ll dig_val = mul(mul(d , pow10[n-i-1]) , temp.cnt );
            ret.sum =  add ( add(ret.sum,dig_val) , temp.sum);
        }
    }
    memo[ru][rd][msk][i] = true;
    return ret;
}
void solve() {
    cin>>l>>r >> k;
    n = r.size();
    while(l.size()!=n)
        l = "0" + l;

    memset(dp,-1,sizeof dp);

    pow10[0] = 1;
    for(int i=1;i<20;i++)
        pow10[i] = mul(pow10[i-1],10);

    cout<<rec(1,1,0,0).sum <<'\n';
}
```

10.5 Digit DP (Kero)

```
void solve()
{
    string l, r;
    int k;
    cin >> l >> r >> k;
    if(r.size() > l.size())
        l = string(r.size() - l.size(), '0') + l;
    k = min(k, 99ll);
    int dp[r.size()][2][2][100][100];
    int n = r.size();
    memset(dp, -1, sizeof(dp));
    function<int(int, int , int , int, int)> sol = [&](int i, int canExceed, int canDown, int
     remNum, int remSum)->int
    {
        if(i == n)
            return !(remSum or remNum);
        auto &ret = dp[i][canExceed][canDown][remNum][remSum];
        if(~ret)
            return ret;
        ret = 0;
        for(int d = 0; d < 10; d++)
        {
            if(!canExceed and d > (r[i] - '0'))
                continue;
            if(!canDown and d < (l[i] - '0'))
                continue;
            ret += sol(i + 1, canExceed | (d != r[i] - '0'), canDown | (d != l[i] - '0'), (
     remNum + (d * (int)powl(10, n - i - 1))) % k, (remSum + d) % k);
        }
        return ret;
    };
    cout << sol(0, 0, 0, 0, 0) << endl;

}
```

10.6 Count Distinct Subsequences

```
dp[0] = 1;
// Traverse through all lengths from 1 to n.
for (int i = 1; i <= n; i++)
{
    dp[i] = (2 * dp[i - 1]) % md;
    // If current character has appeared
    // before, then remove all subsequences
    // ending with previous occurrence.
    if (last[str[i - 1]] != -1)
        dp[i] = (dp[i] - dp[last[str[i - 1]]] + md) % md;
    // Mark occurrence of current character
    last[str[i - 1]] = (i - 1);
}
return dp[n]; // dp[n] - 1 to remove empty subsequence
```

10.7 Largest Zero Submatrix

```
int zero_matrix(vector<vector<int>> a)
{
    int n = a.size();
    int m = a[0].size();
    int ans = 0;
    vector<int> d(m, -1), d1(m), d2(m);
    stack<int> st;
    for (int i = 0; i < n; ++i)
    {
        for (int j = 0; j < m; ++j)
```

```
        {
            if (a[i][j] == 1)
                d[j] = i;
        }
        for (int j = 0; j < m; ++j)
        {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d1[j] = st.empty() ? -1 : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();
        for (int j = m - 1; j >= 0; --j)
        {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d2[j] = st.empty() ? m : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();
        for (int j = 0; j < m; ++j)
            ans = max(ans, (i - d[j]) * (d2[j] - d1[j] - 1));
    }
    return ans;
}
```

10.8 Number of Paths of Length K

```
#include <bits/stdc++.h>
#define Mazen_Alaa ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;
typedef long double ld;

using namespace std;


void solve() {
    int n,m,k;cin>>n>>m>>k;

    Matrix T(n+1,Row(n+1));
    // if u is parent to v => t[u][v] = 1;
    for(int i=0,u,v;i<m;i++){
        cin>>u>>v;
        T[u][v] = 1;
    }
    Row a(n+1);
    for(ll &i:a)
        i = 1;
    Matrix v;
    v.push_back(a);
    T = power(T,k);
    Matrix res = mul(v,T);

    ll tot = 0;
    for(int i=1;i<=n;i++){
        tot += res[0][i];
        if(tot >= mod)
            tot-= mod;
        if(tot < 0)
            tot += mod;
    }
    cout<<tot<<'\n';
}
```

10.9 Sub-Rects with Unique Values

```
vector<vector<int>> height(n, vector<int>(m, 1)); // num of elements top of a[i]
[j] that equal to it
    vector<vector<int>>
        dp(n, vector<int>(m, 0)); // num of subrectangles that
a[i][j] is its bottom right corner
    // setting height for each element
    for (int i = 1; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        if (a[i][j] == a[i - 1][j])
            height[i][j] += height[i - 1][j];
    }
}
for (int i = 0; i < n; i++)
{
    vector<int> st;
    for (int j = 0; j < m; j++)
    {
        int k = -1;
        while (st.size())
        {
            int col = st.back();
            if (a[i][col] != a[i][j] || height[i][col] < height[i][j])
                break;
            st.pop_back();
        }
        // if k = -1: means that all colums behind: same element and their hieght > height[i][
    j]
        // if k = val: means that all columns between [k+1:j] are valid
        // k = val because of: a[i][k]!=a[i][j] or height[i][k] < height[i][j]
        if (st.size())
            k = st.back();
        // each col in [k+1:j] has at least height[i][j], so all of them can end
        at a[i][j] as bot - right corner
                                dp[i][j] = height[i][j] * (j - k);
        if (k != -1)
        {
            // here: k = val because of height[i][k] < height[i][j]
            // so all subrectangles that endes at i,k -> can end at i,j. As
            height[i][k] < height[i][j] if (a[i][j] == a[i][k])
                                dp[i][j] += dp[i][k];
        }
        st.push_back(j);
    }
}
```

11 Game Theory

11.1 Grundy (Sprague-Grundy)

```cpp
const int H = 1e4 + 5;
vector<int> grundy(H, 0);
for (int i = 1; i < H; i++)
{
    set<int> reachable;
    for (auto &move : moves)
        if (i - move >= 0)
            reachable.insert(grundy[i - move]);

    for (auto &x : reachable)
        if (grundy[i] == x)
            grundy[i]++;
        else
            break;
}
```

11.2 Mex Set

```cpp
int sz;
struct Set
{
    set<int> missing;
    Set()
    {
        for(int i = 0; i < sz; i++)
        {
            missing.insert(i);
        }
    }
    int getMex()
    {
        return missing.empty() ? sz : *missing.begin();
    }
    void insert(int x)
    {
        if(missing.count(x))
            missing.erase(x);
    }
    void clear()
    {
        for(int i = 0; i < sz; i++)
        {
            missing.insert(i);
        }
    }
};
```

12 Miscellaneous

12.1 Baby Step Giant Step

```cpp
int d_log(int a, int b, int mod) {
    int n = sqrt(mod) + 1;
    // store b * a ^ q  for all 0 <= q < n
    hash_map<int, int> mp;
    for (int i = 0, cur = b; i <= n ;i++, cur = 1LL * cur * a % mod)
        mp[cur] = i;
    int an = fp(a, n, mod);
    // lets try all a ^ (n * i) == b * a ^ q for all q values using the hash map
    for (int i = 1, cur = an; i <= n; i++, cur = 1LL * cur * an % mod)
        if (mp.find(cur) != mp.end())
                return n * i - mp[cur];
    return -1;
}
```

12.2 Catalan Numbers

```cpp
/*
 * ========================================================================
 * CATALAN NUMBERS: CLASSIC SUB-PROBLEMS
 * The nth Catalan number (C_n) solves several classic combinatorial problems:
 *
 * 1. Valid Parentheses (Dyck Words):
 *    Number of correctly matched bracket sequences consisting of 'n' pairs
 *    of parentheses.
 *    Example (n=3): ((())), (()()), ()()(), (())(), (()()) -> C_3 = 5
 *
 * 2. Binary Search Trees (BSTs):
 *    Number of structurally unique BSTs that can store 'n' distinct keys.
 *
 * 3. Polygon Triangulation:
 *    Number of ways to divide a convex polygon with 'n+2' sides into triangles
 *    by drawing non-intersecting diagonals.
 *
 * 4. Grid Paths (Dyck Paths):
 *    Number of monotonic paths on an n x n grid from bottom-left (0,0) to
 *    top-right (n,n) that never cross above the main diagonal.
 *
 * 5. Non-crossing Chords:
 *    Number of ways to connect 2n points on a circle to form 'n' chords
 *    such that no two chords intersect.
 *
 * 6. Full Binary Trees:
 *    Number of full binary trees (where every vertex has either 0 or 2 children)
 *    that have exactly 'n' internal nodes (and n+1 leaves).
 * ========================================================================
 */

long long catalanMod(int n) {
    if (n <= 1) return 1;

    long long numerator = 1;
    long long denominator = 1;

    // Calculate (2n)! / (n! * n!) % MOD
    for (int i = 0; i < n; i++) {
        numerator = (numerator * (2LL * n - i)) % MOD;
        denominator = (denominator * (i + 1)) % MOD;
    }

    // Multiply numerator by the modular inverse of the denominator
```

```
    long long combinations = (numerator * modInverse(denominator)) % MOD;

    // Multiply by the modular inverse of (n + 1)
    long long result = (combinations * modInverse(n + 1)) % MOD;

    return result;
}

long long catalan(int n) {
    if (n <= 1) return 1;

    long long res = 1;

    // Calculate nCr(2n, n) safely
    // Note: This relies on exact divisibility at each step to avoid early truncation
    for (int i = 0; i < n; ++i) {
        res *= (2LL * n - i);
        res /= (i + 1);
    }

    // Divide by (n + 1) to get the Catalan number
    return res / (n + 1);
}
```

12.3 Compress

```
vector<int> temp = a;
sort(temp.begin(), temp.end());
temp.erase(unique(temp.begin(), temp.end()), temp.end());


for(int i = 0; i < n; i++){
    a[i] = lower_bound(temp.begin(), temp.end(), a[i]) - temp.begin() + 1;
}
```

12.4 Math Operations on Strings

```
string multiply(const string &a, const string &b)
{
    if (a.size() < b.size())
        swap(a, b);
    int n = a.size(), m = b.size();
    vector<int> ans(n + m + 1);
    for (int i = n - 1; ~i; i--)
    {
        for (int j = m - 1; ~j; j--)
        {
            int idx = i + j;
            int cur = (a[i] - '0') * (b[j] - '0');
            ans[idx] += cur;
            if (ans[idx] >= 10)
            {
                if (idx)
                {
                    ans[idx - 1] += (ans[idx] / 10);
                    ans[idx] %= 10;
                }
            }
        }
    }
    string ret;
    int q = 0;
    if (ans[0] >= 10)
    {
        q = 1;
        ret += char('0' + ans[0] / 10);
        ans[0] %= 10;
    }
    int cnt = 1;
    for (int i = 0; i < n + m - 1; i++)
    {
        if (q || ans[i] != 0)
        {
            for (int j = i; j < n + m - 1; j++)
                ret += char('0' + ans[j]);
            cnt = 0;
            break;
        }
    }
    if (cnt)
        ret = "0";
    return ret;
}

string stringStringAddition(string a, string b)
{
    string ret, other;
    if (a.size() > b.size())
        ret = a, other = b;
    else
        ret = b, other = a;
    reverse(ret.begin(), ret.end());
    reverse(other.begin(), other.end());
    int diff = ret.size() - other.size();
    other += string(diff, '0');
    for (int i = ret.size() - 1; i >= 0; i--)
    {
        int cur = (ret[i] - '0') + (other[i] - '0');
        ret[i] = (cur % 10) + '0';
        cur /= 10;
        int currentIdx = i;
        while (cur > 0)
        {
            currentIdx++;
            if (currentIdx == ret.size())
                ret += '0';
            cur += (ret[currentIdx] - '0');
            ret[currentIdx] = (cur % 10) + '0';
            cur /= 10;
        }
    }
    reverse(ret.begin(), ret.end());
    return ret;
}

string longDivision(string num, ll divisor)
{
    string ans;
```

```
    ll idx = 0;
    ll temp = num[idx] - '0';
    while (temp < divisor)
        temp = temp * 10 + (num[++idx] - '0');
    while (num.size() > idx)
    {
        ans += (temp / divisor) + '0';
        temp = (temp % divisor) * 10 + num[++idx] -
               '0';
    }
    if (ans.length() == 0)
        return "0";
    return ans;
}
```

12.5 Ternary Search

```
while (s <= e)
{
    m1 = s + (e - s) / 3;
    m2 = e - (e - s) / 3;
    int ans1 = solve(m1);
    int ans2 = solve(m2);
    ans = min({ans, ans1, ans2});
    if (ans1 > ans2)
        s = m1 + 1;
    else
        e = m2 - 1;
}
```

12.6 2D Partial Sum

```
void update(int x1, int y1, int x2, int y2, int val)
{
    p_sum[x1][y1] += val;
    p_sum[x2 + 1][y2 + 1] += val;
    p_sum[x1][y2 + 1] -= val;
    p_sum[x2 + 1][y1] -= val;
}

for (int i = 1; i < 1004; i++)
{
    for (int j = 1; j < 1004; j++)
    {
        pref[i][j] += pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1];
    }
}
while (q--)
{
    ll la, lb, pa, pb;
    cin >> la >> lb >> pa >> pb;
    cout << pref[pa - 1][pb - 1] - pref[la][pb - 1] - pref[pa - 1][lb] + pref[la][lb] << endl;
}
```

12.7 Stress Test

```
#include <bits/stdc++.h>

using namespace std;

// Random test case generator (writes input to in.txt)
void generate_test_case()
{
    ofstream fout("io/input.txt");
    // TODO : Test Generation
    fout.close();
}

// Read entire file into a string (for output comparison)
string read_file(const string &filename)
{
    ifstream fin(filename);
    stringstream buffer;
    buffer << fin.rdbuf();
    return buffer.str();
}

int main()
{
    srand(time(0));
    int test_num = 0;

    for(; test_num < 1000;)
    {
        test_num++;
        generate_test_case();

        // Run both solutions on the same input
        system("./build/naive");
        system("./build/fast");
        // Compare outputs
        string out = read_file("io/output.txt");
        string brute = read_file("io/output_naive.txt");

        if (out != brute)
        {
            cout << "Test " << test_num << " FAILED!\n";
            // cout << "Input:\n" << read_file("io/input.txt") << "\n";
            // cout << "Fast output:\n" << out << "\n";
            // cout << "Brute output:\n" << brute << "\n";
            break;
        }
        else
        {
            cout << "Test " << test_num << " passed.\n";
        }
    }

    return 0;
}
```